

Article

XAI-IDS: Toward Proposing an Explainable Artificial Intelligence Framework for Enhancing Network Intrusion Detection Systems

Osvaldo Arreche ¹, Tanish Guntur ² and Mustafa Abdallah ^{3,*}

- ¹ Electrical and Computer Engineering Department, Purdue School of Engineering and Technology, Indiana University-Purdue University Indianapolis (IUPUI), Indianapolis, IN 46202, USA; oarreche@iu.edu
- ² Computer and Information Science Department, Indiana University-Purdue University Indianapolis (IUPUI), Indianapolis, IN 46202, USA; tguntur@iu.edu
- ³ Computer and Information Technology Department, Purdue School of Engineering and Technology, Indiana University-Purdue University Indianapolis (IUPUI), Indianapolis, IN 46202, USA
- * Correspondence: mabdall@iu.edu

Abstract: The exponential growth of network intrusions necessitates the development of advanced artificial intelligence (AI) techniques for intrusion detection systems (IDSs). However, the reliance on AI for IDSs presents several challenges, including the performance variability of different AI models and the opacity of their decision-making processes, hindering comprehension by human security analysts. In response, we propose an end-to-end explainable AI (XAI) framework tailored to enhance the interpretability of AI models in network intrusion detection tasks. Our framework commences with benchmarking seven black-box AI models across three real-world network intrusion datasets, each characterized by distinct features and challenges. Subsequently, we leverage various XAI models to generate both local and global explanations, shedding light on the underlying rationale behind the AI models' decisions. Furthermore, we employ feature extraction techniques to discern crucial model-specific and intrusion-specific features, aiding in understanding the discriminative factors influencing the detection outcomes. Additionally, our framework identifies overlapping and significant features that impact multiple AI models, providing insights into common patterns across different detection approaches. Notably, we demonstrate that the computational overhead incurred by generating XAI explanations is minimal for most AI models, ensuring practical applicability in real-time scenarios. By offering multi-faceted explanations, our framework equips security analysts with actionable insights to make informed decisions for threat detection and mitigation. To facilitate widespread adoption and further research, we have made our source code publicly available, serving as a foundational XAI framework for IDSs within the research community.

Keywords: intrusion detection systems; explainable AI; network security; Shapley additive explanations; LIME; CICIDS-2017; NSL-KDD; RoEduNet-SIMARGL2021; feature selection



Citation: Arreche, O.; Guntur, T.; Abdallah, M. XAI-IDS: Toward Proposing an Explainable Artificial Intelligence Framework for Enhancing Network Intrusion Detection Systems. *Appl. Sci.* **2024**, *14*, 4170. <https://doi.org/10.3390/app14104170>

Academic Editors: Rongxing Lu, Sin Gee Teo and Ruitao Feng

Received: 5 April 2024

Revised: 29 April 2024

Accepted: 7 May 2024

Published: 14 May 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The primary goal of an intrusion detection system (IDS) is to identify the unauthorized use, misuse, and abuse of computer network systems by both system insiders and external penetrators [1,2]. Typically, the design of any traditional IDS is based on the belief that an intruder's behavior will be noticeably different from that of a legitimate user and that many unauthorized actions are detectable. With the exponential growth in the number of cyber-intrusions in current networked systems and the high evolution rate of such intrusions, the need to detect such malicious activities and stop related attacks inspires new research directions for developing advanced artificial intelligence (AI) techniques for IDSs [3,4]. Numerous research works have been developed in the literature to detect intrusions, with a focus on statistical anomalies [5,6] and rule-based misuse models [7,8], some of which have also been deployed on an experimental basis in real-world systems [9,10].

However, in the era of widespread cyber connectivity among heterogeneous computer networks, intrusion detection encounters numerous contemporary challenges. Intruders now possess greater access, enabling them to execute increasingly sophisticated attacks without detection [11,12]. This scenario has prompted the exploration of AI's potential in bolstering intrusion detection capabilities within IDSs [13–15]. Various AI techniques have been employed to enhance intrusion detection, encompassing artificial neural networks [16,17], support vector machines [18,19], decision trees [20–22], naive Bayes [23,24], and random forest [25–27].

While prior AI-based approaches for IDSs have acknowledged the influence of individual features on AI decision-making, they often overlook the intricacies of attack-specific feature relevance or the utilization of these features to enhance global and local explanations of AI methods within IDSs. In particular, earlier research has primarily concentrated on the classification accuracy of different AI algorithms, without offering insights into their underlying behavior and decision-making processes. Moreover, they have not explored the importance of attack-specific features, the integration of global and local explanations, or the examination of diverse intrusion datasets and AI model sets. These constraints highlight the urgent need to utilize the relatively new domain of explainable AI (XAI) to enhance the clarity of AI decisions in IDSs [28]. Moreover, other prior works have discussed the proliferation of network threats and cyber-attacks in recent years and emphasized the need for the development of sophisticated IDSs capable of not only detecting but also effectively mitigating such threats [29,30]. Additionally, the integration of deep learning techniques into IDSs has shown promising results in real-time anomaly detection for IoT and SCADA infrastructures [31,32]. This diversified foundation further supports this study's exploration of XAI within network IDSs.

The XAI field offers two primary advantages when applied to contemporary IDSs. Firstly, it fosters trust between the AI agent and the human security agent, which holds paramount importance in high-risk domains such as IDSs. By substantiating AI predictions with evidence, XAI establishes trust with security domain experts. Secondly, XAI enhances transparency by enabling analysts to comprehend why and how the model operates. This demystifies network intrusion detection, particularly in the context of black-box systems. However, it is important to note that the efficacy of this second benefit relies heavily on the quality of the data and their labeling. For instance, if the network intrusion data used to train the model are poorly labeled, it will inevitably impact the performance of black-box XAI methods.

In this paper, we propose an end-to-end XAI framework, termed XAI-IDS, designed to enhance explainability in IDSs. The framework's low-level structure is illustrated in Figure 1. By "end-to-end XAI framework", we specifically mean that we address all stages, including loading the dataset, preprocessing the data, training black-box AI models, generating global and local XAI graphs, extracting top intrusion features based on XAI methods, and providing explanations on local and global scales using the extracted features and XAI graphs. We implemented our framework to examine three network intrusion datasets, each with distinct characteristics. The first two datasets, CICIDS-2017 [33] and NSL-KDD [34], are widely acknowledged in the field of network intrusion detection as testbeds or controlled-environment datasets. In contrast, the RoEduNet-SIMARGL2021 dataset [35] comprises realistic network traffic. Our framework is capable of furnishing both global and local explanations of the AI model. This entails elucidating how features in the network traffic data collectively influence the outcome and explaining the decision-making process of the AI model for specific data instances.

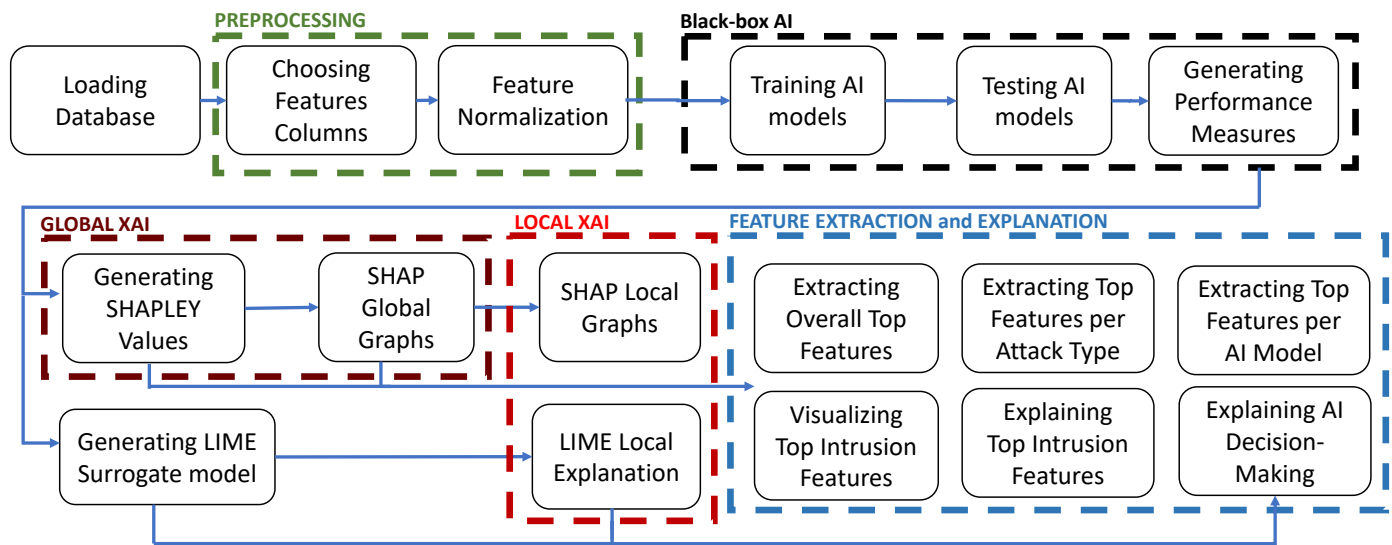


Figure 1. A summary of our XAI framework designed for network intrusion detection.

Furthermore, we demonstrate that our framework furnishes human-understandable explanations for the decision-making of the AI model, in stark contrast to black-box models that offer no justifications for their decisions. Additionally, we illustrate that the benefits derived from XAI explanations do not entail a significant time overhead for generating such explanations for most AI models. Finally, we conduct a benchmark comparison with two prior solutions for feature selection in network intrusion detection systems [35,36]. Our findings reveal that most AI models exhibit superior performance when utilizing the top significant features selected by XAI-IDS compared to baseline methods.

Summary of Contributions: Below, we detail our key contributions:

- We present a comprehensive end-to-end framework designed to improve the explainability of AI models used in network intrusion detection tasks. This framework provides both overarching and detailed explanations for AI models dedicated to intrusion detection.
- We extract important features, both model-specific and intrusion-specific, across various classes of AI models and different types of network intrusions.
- We offer human-understandable explanations and justifications for the decision-making process of the AI model, aiding the security analyst.
- We apply seven black-box AI models to one real-world and two emulated network intrusion datasets, presenting diverse performance metrics.
- We provide our source codes to the community, establishing a foundational XAI framework for network intrusion detection and promoting further exploration with additional datasets and models (the URL for our source codes is https://github.com/ogarreche/XAI_NIDS (accessed on 9 April 2024)).

2. The Problem Statement

Recall that relying on AI for network intrusion detection introduces several challenges, including the variability in performance among different AI models and the opacity of their decision-making processes. Current methods predominantly focus on optimizing accuracy without providing sufficient transparency, leaving security analysts with tools they cannot fully trust or understand [37]. To address these shortcomings, this work proposes a foundational step toward developing an XAI framework specifically designed for network intrusion detection. We aim to provide insights into IDS operations, making the decision-making processes within IDSs understandable. By introducing this framework, we propose a significant shift toward more reliable and interpretable security systems that

stakeholders can robustly audit and trust. In this section, we start by outlining the problem statement to motivate the need for our framework.

2.1. Network Intrusion Types

There are several common types of network intrusions, as categorized within the MITRE ATT&CK framework [38]. In our study, we consider the following main network attacks:

Normal Traffic: This pertains to normal traffic gathered from the network.

PortScan (PS)/Network Service Discovery [MITRE ATT&CK ID: T1046]: In this type of intrusion, the attacker seeks to conduct reconnaissance on the victim's computer, often as a preliminary step to subsequent attacks. PortScan entails dispatching connection requests to multiple ports to pinpoint possible access points for intrusion [39].

Denial of Service (DoS)/Network Denial of Service [MITRE ATT&CK ID: T1498]: This attack type aims to render the target network or system unavailable by inundating it with excessive traffic, causing it to become overwhelmed and inaccessible [35,36].

Brute Force [MITRE ATT&CK ID: T1110]: In this attack, the intruder attempts to gain unauthorized access to the network by systematically trying all possible passwords [36].

Web Attack/Initial Access [MITRE ATT&CK ID: TA0001, T1659, T1189]: These attacks exploit vulnerabilities in web applications to gain initial access to the underlying systems [40].

Infiltration/Initial Access [MITRE ATT&CK ID: TA0001]: This assault entails securing initial entry into a system or application, typically via targeted spear phishing or by exploiting vulnerabilities [41].

Botnet/Compromise Infrastructure [MITRE ATT&CK ID: T1584.005, T1059, T1036, T1070]: Botnets are automated attacks that leverage hijacked devices to execute malicious activities, often orchestrated remotely by the attacker [42].

Probe Attack/Network Scanning or Surveillance [MITRE ATT&CK ID: T1595]: These attacks consist of scanning a network to collect information or identify known vulnerabilities, acting as the initial phase of more extensive attacks [43].

Remote-to-Local Attack (R2L) [MITRE ATT&CK ID: TA0001, T1110, T1078]: R2L attacks occur when an attacker gains unprivileged access to a machine over the network, typically without initially having an account on that machine [43].

User-to-Root Attack (U2R) [MITRE ATT&CK ID: TA0004, T1078]: U2R attacks involve exploiting vulnerabilities to escalate privileges from an unprivileged user account to gain root access to the system [43].

These various types of network intrusions pose significant threats to the security of computer networks. Understanding and effectively detecting these intrusions are critical for maintaining network integrity and security. In our study, we consider these different intrusion types and aim to develop a framework that can accurately detect and provide explanations for them using black-box AI models. By tackling the lack of explainability in IDSs, our work strives to boost the overall efficacy and reliability of intrusion detection systems in practical applications.

2.2. Intrusion Detection Systems

The growing complexity of network attacks presents a serious risk to various essential infrastructures [44,45]. To counteract this, intrusion detection systems (IDSs) are implemented to protect networks from malicious actions conducted by both insiders and external attackers [46]. Conventional IDS frameworks operate under the premise that the activities of an intruder will notably deviate from those of a legitimate user, making many unauthorized actions detectable [47]. Recent developments in AI have paved the way for the creation of AI models that can autonomously detect network intrusions [48].

2.3. Shortcomings of Black-Box AI Models

Although AI models have simplified the process of intrusion detection, their opaque nature poses several challenges. The complex interactions of features within AI models obscure the understanding of how and why specific outcomes are produced. This opacity, typical in AI models such as random forest and deep neural networks, complicates the interpretation of model behavior, especially when errors occur. This issue is particularly crucial in safety-sensitive areas like network security through IDSs, underscoring the importance of elucidating the decision-making processes of AI models [49].

2.4. Explainable AI

Explainable AI (XAI) has developed as a research area dedicated to mitigating the opaque qualities of many AI models. XAI endeavors to clarify which features influence the decision-making process of an AI model, both on a global scale over numerous data instances and locally on individual instances. XAI approaches can be model-agnostic, suitable for any AI model, or model-specific, designed for particular AI models. Techniques involve constructing surrogate models or producing lists of important features for the AI model [50,51].

Main Methodologies of SHAP and LIME: One widely used explainable artificial intelligence (XAI) tool is SHAP [50], which leverages the game theory concept of Shapley values to elucidate an AI model's decisions. SHAP computes Shapley values for each feature by evaluating the prediction using all other features except the one under scrutiny. It then quantifies the impact of each feature's contribution by subtracting the results, thereby determining its relative importance to the prediction. This iterative process assigns a "fair" contribution to each feature using Shapley values, enabling a comprehensive analysis of feature contributions. SHAP facilitates both global and local model analyses and is generally model-agnostic, albeit with some model-specific techniques.

Another prominent XAI tool is Local Interpretable Model-agnostic Explanations (LIME) [51]. LIME enhances interpretability by approximating the AI model locally with an interpretable surrogate model. It provides insights into the model's predictions by creating surrogate models that approximate the behavior of the original model within a specified locality surrounding the analyzed sample. By discerning the features that exert the most influence on the model's output for each prediction, LIME highlights the key contributing features, enhancing the understanding of the model's decision-making process.

2.5. Benefits of XAI for Network IDSs

In the field of network intrusion detection, the absence of detailed explanation systems for black-box-based IDSs obstructs the interpretability of outputs from AI-based IDSs. Achieving high accuracy while elucidating AI algorithm behavior and decision-making is crucial for building trust in AI algorithms among security analysts. XAI frameworks can support analysts by offering explanations for decisions, enabling more effective investigations, and ultimately reducing time spent [52]. Our XAI framework seeks to tackle these challenges, providing deeper insights into AI model predictions and helping analysts comprehend and interpret results from intrusion detection.

3. Materials and Methods

The main goal of this study is to provide an explainable AI (XAI) pipeline that assists security analysts in understanding network traffic features and the decisions made by AI models for intrusion detection, ultimately helping to mitigate various types of network intrusions. To achieve this goal, we present a comprehensive end-to-end XAI framework that includes the components described in the following subsections.

3.1. An End-to-End XAI Pipeline for IDSs

Monitoring and Storing Traffic: The pipeline starts with storing the network traffic by logging the accompanied activities, where such activities can represent normal traffic or a prospective attack on the network. Such logs of the different network activities are stored in a database (DB).

AI Classification and XAI Techniques: The database that contains traffic logs is next used to pipeline its data to train different AI models to detect and identify different possible types of attacks. In such training, we utilize black-box AI models for the intrusion classification task. However, as mentioned earlier, these black-box models would produce results that are not well explained or understood by the security analyst. Therefore, the next essential component in our framework is the application of XAI techniques to provide a diverse set of explanations to the security analyst about different attack types, different intrusion features, and different decisions by the AI models. These explanations (by the XAI model) are then organized in dashboards and intuitive graphs and shown to the security analyst. Given such explanations, the security analyst can investigate specific traffic or make an informed decision by leveraging the XAI explanations, which ultimately enhances the time efficiency of the analyst.

3.2. Low-Level XAI Pipeline Components

We will now detail the fundamental components of our XAI pipeline, as illustrated in Figure 1 and in the Algorithm 1.

Loading Intrusion Database: The first step in our pipeline is to import data from the database. We employ three extensively used network intrusion detection datasets: RoEduNet-SIMARGL2021 [35], CICIDS-2017 [33], and NSL-KDD [34]. These datasets are critical for our analyses and the training of our models.

Choosing Feature Columns: The second component in our framework is the selection of important features from the database. Such feature extraction is essential for building AI models for intrusion detection tasks. The feature extraction process usually impacts AI models. We followed prior works [33,35] for extracting the basic set of features for each network intrusion dataset using information gain [36] and K-best [35] methods. As an important note, for RoEduNet-SIMARGL2021 and CICIDS-2017, the experiments for choosing a basic set of features using K-best and information gain were already performed in their respective works. We also performed K-best and information gain for the NSL-KDD dataset. Furthermore, in this work, we perform feature extraction using XAI methods (particularly SHAP) for all three datasets considered in our work (as explained later in this section). In particular, from the basic set of features (explained above), we extract the top intrusion features for different AI models and intrusion types. Based on this feature selection, we also select the best features among all AI models (as will be shown in our evaluation). We also emphasize that we take advantage of our XAI-based feature selection to generate a new set of features (which we use to compare our approach against baselines, as will be shown in Section 4.4), thus creating a feedback loop in the framework to update the basic features after the whole process can be exploited in our framework when applied to real-world IDSs.

Feature Normalization: To prevent variations in scales among different traffic features, we apply a standard feature normalization step (min-max feature scaling) to all columns in our datasets (where we apply feature scaling for each column, one column at a time, to address inconsistencies across different features' scales). This process ensures that all features are brought to a consistent numerical scale, thereby avoiding any discrepancies in magnitude across the dataset. Such a process has been applied in several related prior works [10,53–55]. In terms of the nature of the datasets, all of them use tabular data. The CICIDS-2017 dataset only has features with numerical values for continuous, discrete, and categorical data. Therefore, we applied normalization to all features used. The RoEduNet-SIMARGL2021 dataset has features with numerical values for discrete and categorical data and three categorical features (i.e., PROTOCOL_MAP, IPV4_DST_ADDR,

and IPV4_SRC_ADDR). However, the feature extractor module for a basic set of features (explained earlier) did not generate two of these three features, and thus, we did not use the IP addresses for any of the experiments. On the other hand, PROTOCOL_MAP has an identical counterpart feature column called PROTOCOL but with the correspondent numeric values that represent each protocol (i.e., TCP is 6, ICMP is 1, gre is 47, ipv6-ICMP is 58, skip is 57, and UDP is 17). For the NSL-KDD dataset, we work with 41 features, of which 4 are categorical. The protocol_type feature has 3 categories (i.e., Protocol_type_tcp, Protocol_type_icmp, and Protocol_type_udp). The service feature is quite diverse, with 70 categories, including service_IRC and service_aol. Flag comes with 11 categories, with examples being flag_S0 and flag_SH. Lastly, the label categorizes network behavior into 23 types, which are further classified into 5 broader categories, which are Normal, DoS, Probe, R2L, and U2R. We discuss feature normalization's effect in Section 4.

Black-box AI Models: After preprocessing the data, we move on to training the AI models. We divide the data into training and testing sets, with 70% used for training and 30% set aside for testing. During this phase, we utilize seven popular AI classification models: 1. RF; 2. DNN; 3. AdaBoost; 4. MLP; 5. KNN; 6. SVM; 7. LightGBM. These models were selected for their popularity and effectiveness in intrusion detection tasks. Random forests and KNN, which are frequently thought to be interpretable in machine learning, also have interpretability issues. Random forests' ensemble nature leads to complicated decision processes that go beyond single-tree clarity [56]. In low-dimensional spaces, KNN's simplicity contrasts with interpretability concerns in high-dimensional spaces, where defining the "nearest" neighbor becomes unclear [57]. These examples highlight the importance of sophisticated explainability approaches (such as the XAI methods considered in this work), even for models that are typically thought to be transparent. Moreover, simple decision trees that have a depth lower than five, for example, and contain a few leaves can be used as an explainability tool, as shown in prior works [20–22]. Since each level of the tree has a clear rule for each node until it reaches a leaf, it shows the model's exact decision process behind every decision. Such an approach is simple to follow and has improved explainability when compared to black-box AI models. However, this comes with much lower performance in terms of accuracy, recall, and precision. For training each model, we configure specific parameters to optimize performance. The detailed settings for each model are presented in Section 4, ensuring that the models are fine-tuned to achieve the best possible performance.

Black-box AI Evaluation: Following model training, the next phase of our framework involves evaluating the performance of each model on unseen test data. For performance analysis, we utilize the following metrics, derived from confusion matrices generated for each model: accuracy (Acc); precision (Prec); recall (Rec); F1 score (F1); Matthews correlation coefficient (Mcc) [58]; balanced accuracy (Bacc); area under the ROC curve (AucRoc). These metrics were chosen based on their widespread usage in similar works focusing on intrusion detection systems (IDSs) [34–36]. Additionally, they allow us to examine the impact of XAI-based feature selection on model performance, enabling comparisons with prior studies on the considered datasets [34–36]. Moreover, we report training and classification times for each AI model, providing insights into computational efficiency alongside performance metrics.

XAI Global Explanations: The aforementioned AI models are considered black-box AI models. Thus, it is essential to provide explanations of such models and accompanied features (traffic log parameters) and labels (attack types). Thus, the next step in our framework is the XAI step. In the first part of this step, we generate global importance values for all features, where we generate different graphs to analyze the impact of each feature on the AI model's decision, which can help in generating expectations about the model's behavior. We utilize different SHAP graphs for global explanations [50], which provide the average marginal contribution of each feature value over all possible coalitions, where a coalition is defined as a combination of features that are used to estimate the Shapley value of a specific feature.

XAI Local Explanations: Our framework consists of two local XAI blocks. First, we use the recent well-known Local Interpretable Model-agnostic Explanations (LIME) [51] to gain insight into what happens inside an AI algorithm by capturing feature interactions. We first generate a model that approximates the original model locally (LIME surrogate model) and then generate the LIME local explanations. Second, we leverage SHAP [50] by generating SHAP local graphs (named waterfall graphs).

Rationale of Choosing SHAP and LIME: We have chosen SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) frameworks for our XAI pipeline. This decision is based on the shortcomings of single-feature explainability methods like partial dependence plots (PDPs), individual conditional expectation (ICE) plots, and leave-one-column-out (LOCO). These single-feature approaches struggle to account for interactions between different features, which can compromise local accuracy. In contrast, SHAP and LIME overcome these challenges by delivering comprehensive explanations that incorporate feature interactions.

Moreover, SHAP utilizes game-theoretic principles to assign importance values to features based on their contribution to predictions. This method pinpoints what influences decisions, thereby enhancing the transparency of the AI's thought process. SHAP is rooted in cooperative game theory, ensuring a consistent and equitable attribution of feature importance [59]. Conversely, LIME offers insights into individual predictions by locally simplifying the model's decision boundary, aiding users and developers in understanding why specific decisions were made. In particular, LIME clarifies individual predictions by perturbing the input data and observing how the predictions change. It constructs a locally interpretable model near the prediction to approximate the complex model's decision boundary, revealing how each feature influences the prediction on a local scale [60]. Both methods play a crucial role in increasing the accountability of AI systems, showcasing their adaptability across various industries [59,61].

The performance of SHAP and LIME varies with the characteristics of the dataset. For example, SHAP tends to perform better with sparse data due to its inherent completeness constraint, which acts as a regularizing factor. This attribute helps stabilize the explanations that SHAP generates, leading to more consistent and reliable interpretations in such cases [62]. SHAP's foundation in well-established game theory principles provides a robust basis for its explanations, enhancing its reliability and trustworthiness [60]. Therefore, SHAP and LIME significantly enhance the interpretability of AI models, which is vital for building user trust and ensuring compliance with regulatory standards. This makes them well suited for our XAI pipeline, enabling accurate and interpretable explanations of model decisions at both the global and local levels.

Feature Explanation: The final component in our framework is the extraction of detailed metrics from the global explanations. In particular, we extract the model-specific features (i.e., top important features for each AI model) and intrusion-specific features (i.e., top features for each intrusion type) for different classes of AI models that we have and different types of network intrusions. We also provide human-understandable explanations of the decision-making of the AI model and the related top features.

XAI Framework Algorithm: We now provide the main steps of our XAI framework (Algorithm 1).

Algorithm 1: XAI Framework Algorithm: We now provide the main steps of our XAI framework

- 1: Load one of the network intrusion datasets (i.e., RoEduNet-SIMARGL2021, CICIDS-2017, or NSL-KDD).
 - 2: Choose feature columns for analysis.
 - 3: Normalize features to prepare data for modeling.
 - 4: Train AI models on the preprocessed data (including ADA, DNN, LightGBM, SVM, MLP, RF, and KNN).
 - 5: Test the AI models to evaluate performance.
 - 6: Generate performance measures to assess model accuracy and reliability (accuracy, recall, F1, precision, BACC, MCC, and AUCROC).
 - 7: **For a global perspective, perform the following steps:**
 - 8: Generate Shapley values for different intrusion features to obtain global explanations of the AI model's behavior.
 - 9: Generate SHAP Global Graphs (SHAP summary plots).
 - 10: Extract top features that influence model decisions:
 - 11: Extract top features specific to each AI model.
 - 12: Extract overall top features across all models.
 - 13: Extract top features specific to each attack type.
 - 14: Visualize and explain top intrusion features from a global perspective.
 - 15: Provide detailed explanations for AI decision-making using SHAP.
 - 16: **For a local perspective, perform the following steps:**
 - 17: Generate a LIME surrogate model to provide local explanations for individual predictions.
 - 18: Visualize and explain top intrusion features using LIME local graphs.
 - 19: Generate Shapley values to obtain local explanations of the model's behavior.
 - 20: Generate SHAP Local Graphs (SHAP waterfall graphs).
 - 21: Provide detailed explanations for AI decision-making using both SHAP and LIME techniques on both local and global scales.
-

3.3. Feature Importance for Different Attacks

One of the important outcomes of our framework is a list of the top features for each attack type. This can help the security analyst look for specific features within the log when investigating suspicious traffic. In our current work, we analyze ten possible classes, which are normal traffic along with nine types of attacks (DoS, PortScan, Brute Force, Infiltration, Web attack, Bot, Probe, U2R, and R2L). We focus on gaining a more in-depth understanding of the feature importance for each attack for the three considered datasets (CICIDS-2017, RoEduNet-SIMARGL2021, and NSL-KDD). Note that the RoEduNet-SIMARGL2021 dataset contains only data for DoS and Port Scan attacks (along with normal traffic), CICIDS-2017 has data for Brute Force, Infiltration, Web attack, and Bot attack classes, and NSL-KDD has only DoS, Probe, U2R, and R2L attack classes.

3.4. Illustrations of Local Explanations by XAI

We now provide two illustrative examples of XAI local explanations.

Local Explanation Using LIME: Figure 2 shows an example of a local explanation of a Port Scan attack sample using LIME for random forest applied to the RoEduNet-SIMARGL2021 database. There are three vertical bars, one for each possible class. These bars contain the top five features with the greatest contributions to this outcome, where the contributions can be seen as five horizontal bars that go toward one classification or another. Here, intrusion features that push the prediction probability toward a specific class (for example, toward Port Scanning) are deemed to contribute positively (i.e., these features are assigned positive contribution values, represented by the bars that are positioned on the right-hand side). On the other hand, those that push the

explanation away from the class are considered to contribute negatively (i.e., features that are assigned negative contribution values, represented by the bars that are positioned on the left-hand side) to the Port Scanning classification. The contribution of each feature is shown in Figure 2 by the horizontal bar positioned below each possible label. Moreover, the prediction probabilities field (top left) shows how much confidence LIME has in its classification, ranging from zero to one. Lastly, the green box shows the actual value for each feature in this particular instance. Based on the observations provided by LIME, an analyst could see which features contributed the most to a certain prediction and also check the confidence score. For example, it could be the case that LIME has prediction scores of 50% for port scanning, 40% for DoS, and 10% for normal traffic. In this case, the security analyst can check for port scanning and DoS since they have close scores.

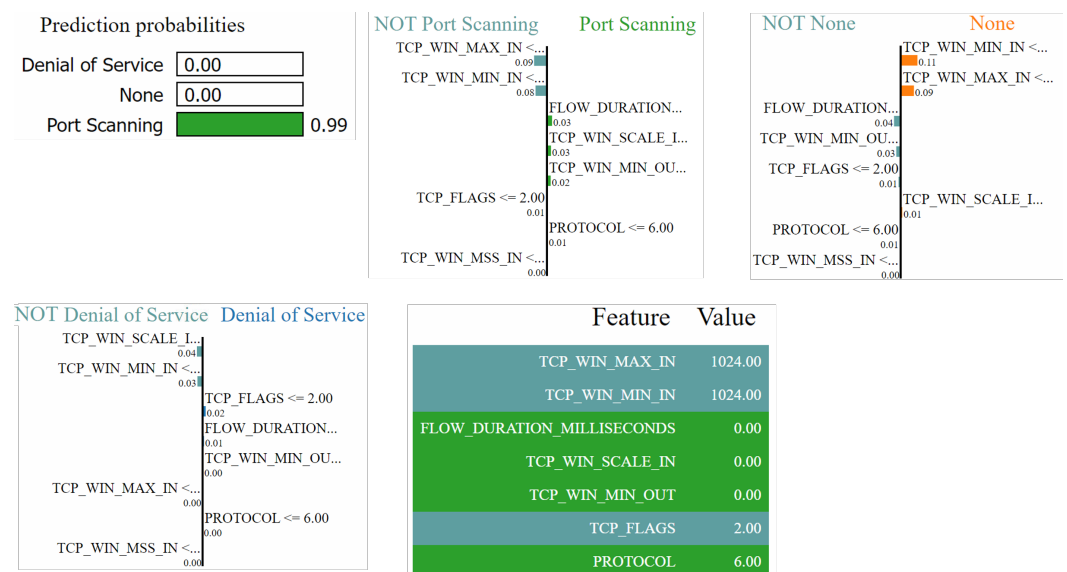


Figure 2. An example of a local explanation of an RF model using LIME on a port scanning attack on the RoEduNet-SIMARGL2021 dataset.

Local Explanations using SHAP: Figure 3 provides an insightful local explanation for a DoS (Denial of Service) instance using SHAP applied to a random forest model. This illustration, structured as a waterfall graph, effectively ranks the most crucial features influencing the model's decision for a specific data sample. The features are listed in descending order of impact, offering a detailed view of their contributions. One important key to this explanation is the concept of a "base value", which, in this context, stands at 0.334. From this baseline, the graph showcases how each feature's value (depicted in gray) either steers the model's decision toward classifying the instance as a DoS attack or away from this classification. The arrows stem from this base value and are pivotal in understanding SHAP's output. They quantitatively indicate the extent to which each feature influences the model's verdict. Features that nudge the prediction toward identifying the instance as DoS are illustrated with arrows extending left from the base value. These are shown in pink and are interpreted as having a positive contribution. Conversely, features that push the decision away from a DoS classification and, thus, potentially toward a non-DoS category extend right from the base value, depicted in blue, indicating a negative contribution. To summarize, each feature's contribution is quantified using SHAP in such a way that the evaluation can determine its contribution to deciding in favor of or against the classification of DoS. Specifically, as shown in Figure 3, features that push the forecast toward DoS (i.e., the right-hand side of the base value with the color magenta) have a positive effect, while those that push the prediction away (i.e., the left-hand side of the base value with the color blue) have a negative effect.

This derives explanations for why this sample was classified as a DoS attack.

Comparing LIME and SHAP for Local Explanations: The LIME local explanation provides more details about the features that affected the decision for each intrusion class, along with the overall prediction probabilities for each class. On the other hand, SHAP's local feature visualization shows the contribution of each top feature to the AI model's decision. From our experiments, we recommend combining complementary local explanations from LIME and SHAP to provide more confident local explanations for an IDS. We also provide examples of local explanations for samples from the CICIDS-2017 and NSL-KDD datasets in Appendix B.

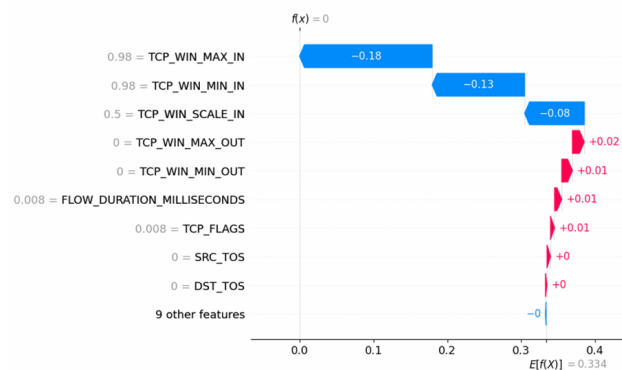


Figure 3. An example of a local explanation for correctly predicting a DoS sample from the RoEduNet-SIMARGL2021 dataset for the RF model using SHAP.

3.5. Top Intrusion Features List

We provide a detailed list of the most significant network intrusion features along with their explanations for the three datasets under investigation. Specifically, explanations for each feature in the RoEduNet-SIMARGL2021 dataset are detailed in Table 1, descriptions of the features for the CICIDS-2017 dataset are included in Table 2, and the features for the NSL-KDD dataset are itemized in Table 3. It is essential to note that our XAI framework will be employed to analyze various metrics regarding the importance of these features in our evaluation (Section 4). We stress that these tables contain explanations for all top intrusion features extracted in our evaluation. This encompasses features selected by baseline methods utilizing information gain or K-best (as per [35,36]), model-specific features derived from SHAP global summary plots, overall top features common across AI models (illustrated in Table 4 based on frequency analysis from SHAP plots), and attack-specific features. Attack-specific features are obtained through manual rearrangement and frequency analysis, focusing on individual attacks rather than a holistic view. Details regarding overall feature importance and attack-specific features are thoroughly discussed in our evaluation in Section 4.3.

Table 1. Description of main features for RoEduNet-SIMARGL2021 dataset [63].

RoEduNet-SIMARGL2021 Features	Explanation
FLOW_DURATION_MILLISECONDS	Flow duration in milliseconds
PROTOCOL_MAP	IP protocol name (tcp, ipv6, udp, icmp)
TCP_FLAGS	Cumulation of all flow TCP flags
TCP_WIN_MAX_IN	Max TCP Window (src->dst)
TCP_WIN_MAX_OUT	Max TCP Window (dst->src)
TCP_WIN_MIN_IN	Min TCP Window (src->dst)
TCP_WIN_MIN_OUT	Min TCP Window (dst->src)
TCP_WIN_SCALE_IN	TCP Window Scale (src->dst)
TCP_WIN_MSS_IN	TCP Max Segment Size (src->dst)
TCP_WIN_SCALE_OUT	TCP Window Scale (dst->src)
SRC_TOS	TOS/DSCP (src->dst)
DST_TOS	TOS/DSCP (dst->src)
FIRST_SWITCHED	SysUptime of First Flow Packet
LAST_SWITCHED	SysUptime of Last Flow Packet
TOTAL_FLOWS_EXP	Total number of exported flows

Table 2. Description of the main features for the CICIDS-2017 dataset [64].

CICIDS-2017 Features	Explanation
Packet Length Std	Standard deviation length of a packet
Total Length of Bwd Packets	Total size of a packet in the backward direction
Subflow Bwd Bytes	Average number of bytes in a backward subflow
Destination Port	Destination Port Address
Packet Length Variance	Variance length of a packet
Bwd Packet Length Mean	Mean size of a packet in the backward direction
Avg Bwd Segment Size	Average size observed in the backward direction
Bwd Packet Length Max	Maximum size of a packet in the backward direction
Init_Win_Bytes_Backward	Total number of bytes in the initial backward window
Total Length of Fwd Packets	Total packets in the forward direction
Subflow Fwd Bytes	Average number of bytes in a forward subflow
Init_Win_Bytes_Forward	Total number of bytes in the initial forward window
Average Packet Size	Average size of a packet
Packet Length Mean	Mean length of a packet
Max Packet Length	Maximum length of a packet

Table 3. Description of main features for the NSL-KDD dataset [34].

NSL-KDD Features	Explanation
duration	Length of the connection
protocol_type	Type of protocol (e.g., TCP, UDP)
service	Network service on the destination (e.g., HTTP, FTP)
flag	Normal or error status of the connection
src_bytes	Number of data bytes from the source to the destination
dst_bytes	Number of data bytes from the destination to the source
logged_in	1 if successfully logged in; 0 otherwise
count	Number of connections to the same host as the current connection in the past two seconds
srv_count	Number of connections to the same service as the current connection in the past two seconds
dst_host_same_srv_rate	Rate of connection to the same service on the destination host
dst_host_srv_count	Count of connections to the same service on the destination host
dst_host_same_src_port_rate	Rate of connections to same source port on the destination host
dst_host_serror_rate	Rate of connections that have activation flags indicating various types of errors
dst_host_rerror_rate	Rate of connections that have REJ (rejected) flags

Table 4. An overall ranking of feature importance (in descending order) across all AI models for the RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD datasets.

Feature Rank	CICIDS-2017	RoEduNet-SIMARGL2021	NSL-KDD
1	Destination Port	TCP_WIN_MSS_IN	dst_host_same_srv_rate
2	Init_Win_Bytes_Backward	TCP_WIN_SCALE_IN	dst_host_srv_count
3	Packet Length Std	TCP_WIN_MAX_IN	dst_host_same_src_port_rate
4	Bwd Packet Length Mean	TCP_WIN_MIN_IN	logged_in
5	Total length of Bwd Packets	TCP_WIN_WIN_OUT	dst_host_serror_rate
6	Packet Length Mean	TCP_FLAGS	count
7	Subflow Backward Bytes	PROTOCOL	srv_count
8	Packet Length Variance	FLOW_DURATION_MILLISECONDS	dst_host_rerror_rate

4. Results

We proceed to present our detailed evaluation results, aiming to address several key questions: (1) What is the performance of black-box AI models on the three considered network intrusion datasets? (2) How does XAI help in understanding and explaining the performance of different AI models for intrusion detection? (3) What are the main features for each AI model? (4) What are the main features for each intrusion type? (5) What are the common influential intrusion features among different types of AI models? (6) What is the time overhead of generating XAI explanations? and (7) How does XAI-based feature selection compare to that in baseline methods?

4.1. Dataset Description

RoEduNet-SIMARGL2021 Dataset [35]: Sourced from the SIMARGL project funded by the European Union, this dataset comprises realistic network traffic and features derived from actual traffic flows. It adheres to the Netflow protocol for organization.

The dataset follows a structured data schema based on Netflow [65], a network protocol devised by CISCO for the collection and analysis of network flows.

CICIDS-2017 Dataset [33]: Developed by the Canadian Institute for Cybersecurity, this benchmark dataset for intrusion detection includes six distinct attack scenarios: Brute Force, heartbleed, botnet, DoS, PortScan, Web attack, and Infiltration. It simulates realistic background traffic using the B-Profile system [66].

NSL-KDD Dataset [34]: An improvement on the original KDD dataset, created by the University of New Brunswick and the National Research Council of Canada, the NSL-KDD dataset features a diverse array of attack types and normal instances, organized according to a data schema that processes raw traffic. This dataset is extensively utilized for testing intrusion detection systems [67,68].

Summary and Statistics of the Datasets: Detailed statistics for each dataset, including its size, the variety of attack types, and the distribution of attack labels, can be found in Table 5. This table offers a comprehensive view of the datasets' compositions and features.

Table 5. A summary of the basic statistics and distribution of samples among different attack types for the three datasets.

(a) Basic statistics of datasets										
Dataset	No. of Labels			No. of Features			No. of Samples			
CICIDS-2017	7			78			2,775,364			
RoEduNet-SIMARGL2021	3			29			31,433,875			
NSL-KDD	5			41			148,517			
(b) Distribution of samples among different attack types										
Dataset	Normal	DoS	PortScan	Brute Force	Web Attack	Bot	Infiltration	Probe	U2R	R2L
CICIDS-2017	84.442%	9.104%	5.726%	0.498%	0.157%	0.071%	0.001%	-	-	-
RoEduNet-SIMARGL2021	62.20%	24.53%	13.27%	-	-	-	-	-	-	-
NSL-KDD	48%	35%	-	-	-	-	-	10.05%	0.47%	6.48%

4.2. Experimental Setup

Computing Resources: Our experiments utilized a high-performance computer (HPC) equipped with four NVIDIA A100 GPUs and 64 GPU-accelerated nodes (NVIDIA, Santa Clara, CA, USA), each with 256 GB of memory, alongside a 64-core AMD EPYC 7713 processor (AMD, Santa Clara, CA, USA) (2.0 GHz, 225 watts). This setup delivered a peak performance of about 7 petaFLOPs.

Coding Tools: For our development, we chose the Python programming language to take advantage of various open-source resources and maintain transparency in our implementation. Our toolkit included Keras and ScikitLearn for AI modeling. Additionally, we integrated XAI toolboxes, such as the following:

- (a) **SHAP [50]:** Employed for generating feature explanations, this tool was instrumental in providing both local and global insights into our AI models for intrusion detection.
- (b) **LIME [51]:** We used LIME to develop surrogate models that approximate the decisions of our AI models on individual samples, helping in crafting detailed local explanations.
- (c) **Other toolboxes:** Tools like Pandas and Matplotlib were also utilized for data manipulation and visualization tasks.

Table 7. Overall performance of AI models with top 15 features versus all features (no feature selection) for the CICIDS-2017 dataset.

AI Model	Top 15 Features							All Features						
	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc
RF	0.98	0.93	0.93	0.93	0.96	0.92	0.99	0.99	0.96	0.96	0.98	0.98	0.94	0.99
ADA	0.99	0.99	0.99	0.99	0.99	0.98	0.95	0.97	0.89	0.89	0.89	0.94	0.88	0.96
DNN	0.95	0.83	0.83	0.83	0.90	0.80	0.96	0.98	0.94	0.94	0.94	0.97	0.94	0.99
SVM	0.97	0.91	0.91	0.91	0.94	0.89	0.75	0.96	0.87	0.87	0.87	0.92	0.85	0.93
KNN	0.99	0.97	0.97	0.97	0.98	0.96	0.89	0.99	0.97	0.97	0.97	0.98	0.96	0.89
MLP	0.97	0.91	0.91	0.91	0.94	0.89	0.98	0.99	0.98	0.98	0.98	0.99	0.98	0.98
LightGBM	0.98	0.94	0.94	0.94	0.96	0.93	0.99	0.98	0.95	0.95	0.95	0.97	0.95	0.99

Table 8. Overall performance of AI models with top 15 features versus all features (no feature selection) for the NSL-KDD dataset.

AI Model	Top 15 Features							All Features						
	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc
RF	0.88	0.54	0.50	0.43	0.75	0.61	0.60	0.84	0.78	0.42	0.36	0.68	0.32	0.62
ADA	0.85	0.46	0.40	0.31	0.64	0.25	0.57	0.81	0.47	0.47	0.21	0.63	0.27	0.58
DNN	0.86	0.60	0.46	0.48	0.68	0.40	0.69	0.88	0.51	0.48	0.46	0.69	0.40	0.69
SVM	0.86	0.54	0.35	0.41	0.50	0.33	0.70	0.87	0.60	0.43	0.43	0.66	0.36	0.70
KNN	0.81	0.60	0.34	0.34	0.62	0.30	0.73	0.88	0.52	0.54	0.73	0.72	0.47	0.73
MLP	0.84	0.57	0.44	0.37	0.66	0.33	0.65	0.90	0.75	0.54	0.72	0.51	0.33	0.68
LightGBM	0.86	0.41	0.47	0.48	0.67	0.34	0.64	0.84	0.39	0.41	0.37	0.68	0.26	0.64

The CICIDS-2017 dataset detailed in Table 7 reveals ADA and KNN as top performers with 0.99 accuracy for all features, while the worst performer is DNN in this case. When using the top 15 features, SVM and KNN perform poorly relative to the other models, particularly in their AUC-ROC scores. MLP is the top performer with the best overall results, followed by RF.

Table 8's analysis of the NSL-KDD dataset highlights MLP as the best performer when all features are considered, particularly excelling in accuracy and F1 score, followed by KNN as a close second. ADA appears as the lesser model for all features, followed by RF and LightGBM in second place with similar performance. When considering the top 15 features, RF is the best performer, followed by DNN, while the weakest performer is KNN, with MLP in second, when considering the respective accuracies.

These findings suggest that RF, MLP, and DNN are robust across various datasets, indicating broad applicability regardless of dataset specifics. LightGBM's performance variability suggests a potential dependence on having a richer feature set for optimal results. On the other hand, SVM, KNN, and ADA seem to benefit from feature selection. This feature sensitivity suggests that tailored feature engineering may be necessary to enhance model effectiveness. Holistically, comparing Tables 6–8 indicates that using feature selection is beneficial, since it is often close to or surpasses the “All Features” setup regarding the results.

Impact of Feature Selection on Black-box AI Models: In this section, we analyze the overall performance of our various black-box AI models for intrusion detection systems (IDSs) with no feature selection. For the RoEduNet-SIMARGL2021 dataset, Table 6 presents the performance metrics of our different AI models. Notably, three models (ADA, SVM, and KNN) exhibit worse performance when trained with all features, whereas only LightGBM demonstrates improved performance without feature selection. Conversely, the other models (DNN, RF, and MLP) show no significant difference in their performance.

For the CICIDS-2017 dataset, as shown in Table 7, four AI models display enhanced performance when trained with all features (RF, DNN, MLP, and KNN), while the remaining models exhibit decreased performance. This contrasts with the observations from the RoEduNet-SIMARGL2021 dataset.

The performance metrics of the AI models on the NSL-KDD dataset, depicted in Table 8, reveal notable differences compared to the results obtained on the RoEduNet-SIMARGL2021 and CICIDS-2017 datasets. In particular, for the NSL-KDD dataset, using the top 15 features instead of the full set generally results in improved performance across most models, indicating a higher sensitivity to feature selection. This trend is especially prominent in the ADA, SVM, KNN, and MLP models. Conversely, the RF and LightGBM models exhibit slightly different trends across the three datasets, with RF performing better with the full feature set and LightGBM maintaining comparable performance regardless of the feature set size. This behavior underscores the significance of feature selection and the varied responses different AI models might exhibit concerning feature complexity.

This experiment highlights the importance of meticulously selecting features to enhance the performance of AI models and underscores that such a feature selection process depends on both the dataset's characteristics and the type of AI model employed.

We emphasize that while these results showing high-level performance provide valuable insights, they do not offer detailed information about each AI model's performance on different classes of attacks, which we explore further in the subsequent sections.

Variation in Best AI Model across Attack Types: Table 9 displays the accuracy for each attack type across the three datasets. The table reveals that most AI models do not consistently achieve superior accuracy across all different attack classes within each dataset (except for SVM on the first dataset and ADA on the second dataset, although SVM exhibits the drawback of a very long training time). This contradicts claims suggesting that a single AI algorithm can outperform others for different datasets with varying intrusion types, underscoring the necessity for an effective approach to understanding how each AI model makes decisions for different attack types, which we elucidate in our current work.

Table 9. Accuracy per attack type for the RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD datasets with 15 features. The best results are in bold text.

AI Model	RoEduNet-SIMARGL2021			CICIDS-2017						NSL-KDD					
	Normal	DoS	PS	Normal	DoS	B. Force	Web	Infilt.	Bot	PS	Normal	DoS	Probe	R2L	U2R
RF	0.99	0.99	0.99	0.99	0.99	0.99	0.98	0.94	0.94	0.99	0.80	0.89	0.83	0.87	0.99
ADA	0.89	0.89	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.72	0.82	0.90	0.87	0.95
DNN	0.96	0.99	0.89	0.96	0.99	0.89	0.84	0.99	0.94	0.99	0.73	0.84	0.91	0.87	0.99
SVM	1.00	0.99	0.99	0.99	0.91	0.99	0.99	0.99	0.91	0.99	0.63	0.73	0.86	0.86	0.99
KNN	0.99	0.99	0.99	0.97	0.99	0.99	0.97	0.99	0.99	0.99	0.72	0.74	0.77	0.87	0.99
MLP	0.99	0.99	0.99	0.96	0.99	0.99	0.98	0.93	0.94	0.99	0.74	0.75	0.84	0.88	0.99
LightGBM	0.89	0.64	0.55	0.99	0.95	0.99	0.95	0.99	0.94	0.99	0.75	0.87	0.82	0.87	0.99

In analyzing the variation in AI model performance across different attack types, as shown in Table 9, it is evident that no single model consistently excels across all conditions (i.e., no model achieves 0.99 in all metrics). SVM stands out for the RoEduNet-SIMARGL2021 dataset, achieving perfect accuracy scores in categories like Normal, DoS, and PS attacks, followed closely by RF, KNN, and MLP. This highlights these models' capability to effectively classify various attack types. Conversely, ADA demonstrates robust performance on the CICIDS-2017 dataset, with high accuracy across all attack types, suggesting its potential as a versatile and reliable model for intrusion detection in this case. Nonetheless, RF, SVM, and KNN come in close second, with 0.99 accuracy in most attacks. For NSL-KDD, RF is the best performer, achieving the highest score for three attack types out of five. On the other end of the spectrum, SVM and LightGBM show weaker performance in specific scenarios. SVM struggles with Normal and DoS classification on the NSL-KDD dataset, indicating potential limitations in handling complex

or infrequent attack vectors. LightGBM's underperformance, particularly in dealing with DoS and PortScan attacks in the RoEduNet-SIMARGL2021 dataset, further stresses the challenges some models face with specific types of attacks or dataset characteristics.

The disparities in model performance across various datasets and attack types show an opportunity to combine their strengths depending on the scenario or context, probably leading to an ensemble learning application in future research. Nonetheless, RF showed great consistency among all datasets, being among the top performers, especially in the NSL-KDD scenario, where it outperforms the other models by a considerable margin, except in Probe and R2L attacks.

Overall Feature Importance across different AI Models based on XAI: We emphasize that such attack-type performance results do not give more insights into the impact of different features on the decisions of the AI models, which we present in our next experiments.

Enhancement of Explanation of AI Decision-Making in XAI-IDS: We first show that our framework provides a human-understandable explanation to the human security analyst for the decision-making of the AI model. On the other hand, black-box AI models do not provide any justification for their decision-making. Figure 4 shows an example from the RoEduNet-SIMARGL2021 dataset. Appendix D shows different examples (explanations) for all datasets. Note that we do not produce the explanations in Figure 4 using an automated code. However, one possible future extension in this direction is to use generative AI to generate such explanations automatically given the used XAI tool (SHAP or LIME), a network traffic sample, and the output from the XAI tool (top features and their contributions). We emphasize that the raw input features in the top part of Figure 4 are not user-friendly for the average user. Hence, in a future development phase, they could be parameterized as features that can be accessed by advanced users or used if further investigation is needed. Another reason for the current layout of Figure 4 is to exemplify the benefits of adding an XAI technique. On the left-hand side is the limited information that black-box AI can provide, which is the probability of the classification outcomes. However, when combined with the XAI technique, we can provide the rule extraction from each class, the contribution from each class to a given outcome, and the list of the top most influential features with their numerical contribution scores. This added value can help non-expert or new security analysts, who could benefit from such explanations.

The Importance of Features via Explainable AI: In this section, we demonstrate the crucial features that influence the decision-making of each AI model used in our study across the three analyzed network intrusion datasets.

- (a) **Global Summary Plot:** We display global summary plots for various AI models, illustrating the most impactful features in the decision-making process of the models. These plots prioritize the importance of features from the most to the least influential. Figures 5 and 6 reveal the feature importance for different AI models within the RoEduNet-SIMARGL2021 and CICIDS-2017 datasets, respectively. Moreover, the global summary plot for the NSL-KDD dataset is detailed in Figure A6 in Appendix E. The importance of each feature is determined by averaging the mean Shapley values across all features. Notably, these plots also highlight the significance of different features across various attack types, represented by distinct colors for each attack label.

Interpretation of Global Summary Plots: Figures 5 and 6 provide insights from the SHAP global explanations. These visual representations represent attacks as stacked horizontal bars, where the length of each bar reflects the magnitude of its SHAP score or its contribution to the model's predictions. Each row in these graphs corresponds to a specific feature and is ranked by the importance of the feature. The various colors within each bar denote different attack classes, showing the impact of each class on the significance of the feature. For example, a long segment of color within a feature's bar suggests that the corresponding attack class plays a substantial role in the model's evaluation of that feature. This detailed portrayal helps us not only understand the overall global significance

of each feature for the AI model but also appreciate the distinct contribution of each feature to individual attack classes. Ultimately, these visuals assist in extracting both the top model-specific features and the top attack-specific features. Further visualizations, such as the waterfall graphs and beeswarm plots in Figures A7 and A8, can also provide analysts with a deeper understanding of the relationships between raw values and SHAP values.

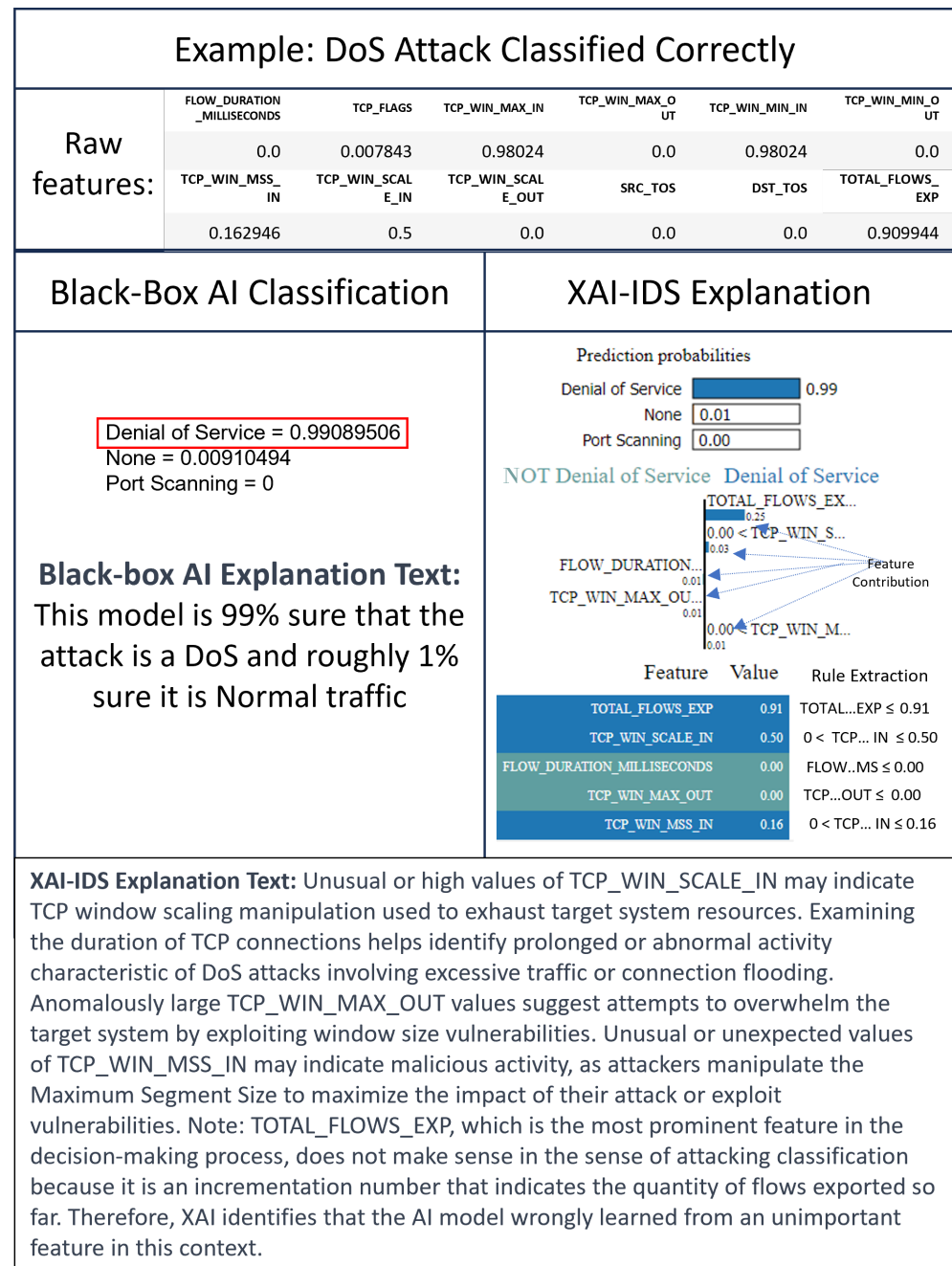


Figure 4. An example of the enhancement of the explanation of the AI model's decision-making using our XAI framework versus a black-box AI model.

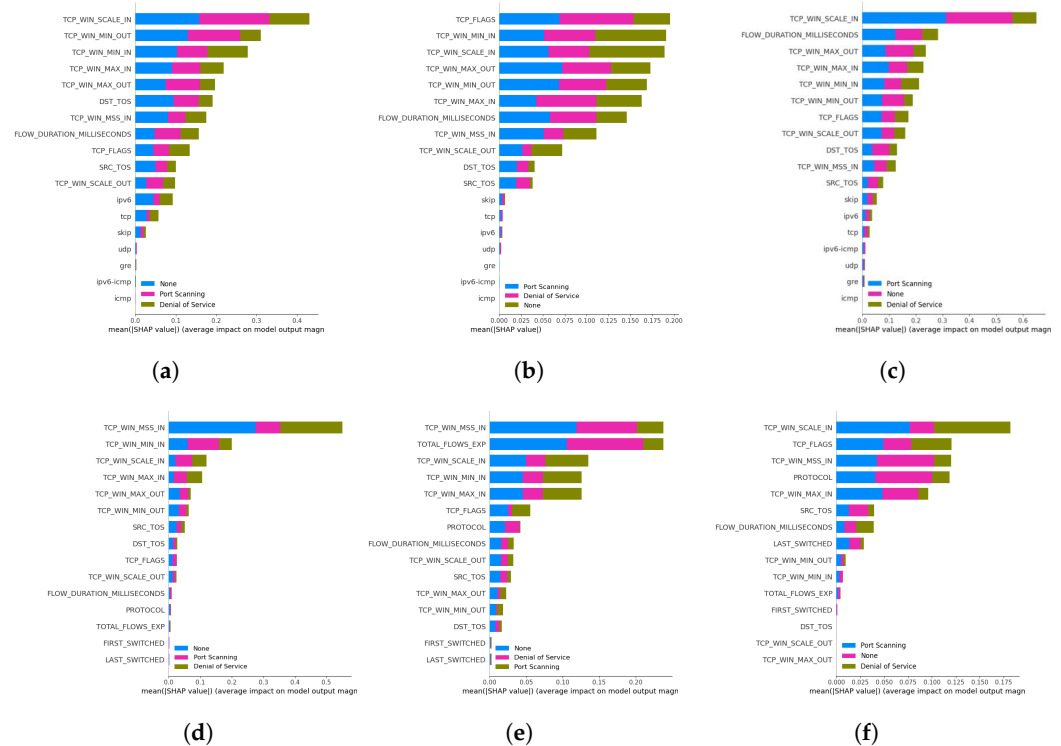


Figure 5. Feature significance determined by the global summary plot (via SHAP) for each intrusion detection AI model utilized with the RoEduNet-SIMARGL2021 dataset. This plot also delineates the importance of each feature according to the attack type, represented by various colors. (a) DNN; (b) RF; (c) LightGBM; (d) SVM; (e) KNN; (f) ADA.

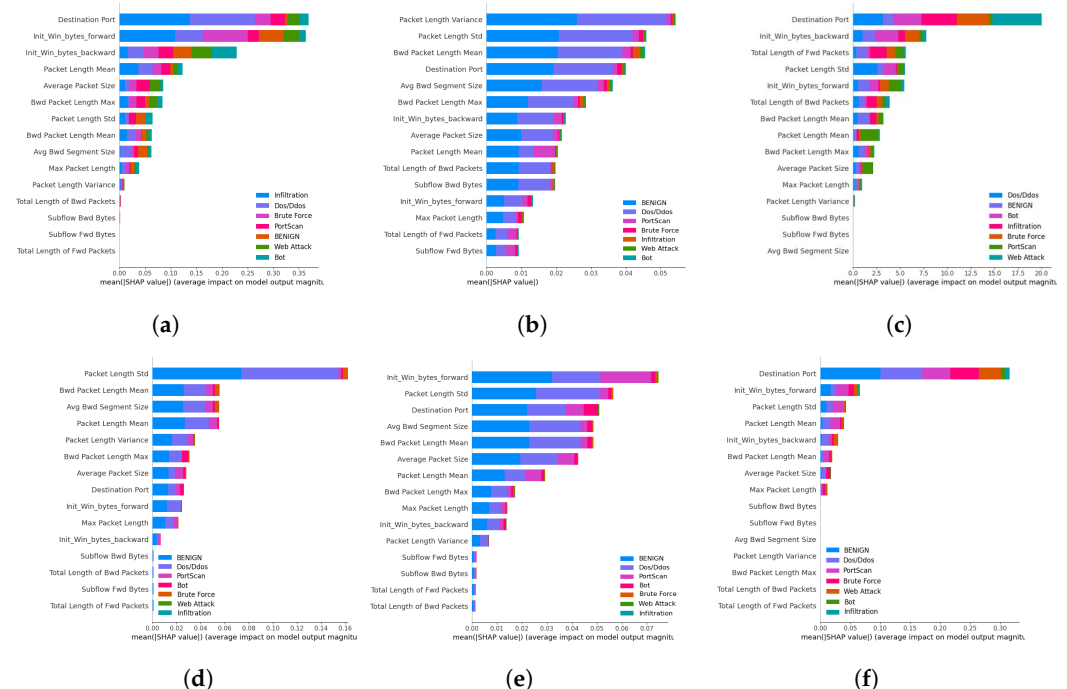


Figure 6. Feature significance illustrated through a global summary plot (created using SHAP) for each AI model designed for intrusion detection with the CICIDS-2017 dataset. This visualization also highlights the importance of each feature by attack type, indicated by varying colors. (a) DNN; (b) RF; (c) LightGBM; (d) SVM; (e) KNN; (f) ADA.

- (b) **Global Beeswarm Plot:** Another visualization that we employ is the global beeswarm plot, which displays various influential features. In contrast to the global summary plot, this plot provides a heatmap indicating the correlation between these features' values and the AI model's predictions (whether the features' values are proportional or inversely proportional to the AI's decision).

In beeswarm plots (Figures A7 and A8), each data instance is represented by a dot on the plot, with the corresponding intrusion features on the left-hand side. The SHAP value of each point determines its horizontal placement, with higher values placed farther away from the center. Additionally, points with a high density are vertically stacked. By observing the distribution of these points, an analyst can gain insight into how a feature may impact the prediction.

The color bar on the right-hand side of the plot indicates the raw value of each instance, with high values shown in red and low values in blue. Consequently, the analyst can comprehend the correlation between SHAP values and raw values by analyzing both the distribution and color. For example, TCP_WIN_MSS_IN (TCP window size) is the most important intrusion feature, on average, for SVM, with higher values indicating the possibility of an attack.

For brevity, Appendix E includes beeswarm plots for our three datasets.

Overall Feature Importance across Different AI Models Based on XAI: Upon evaluating the feature importance within each intrusion detection AI model, we identify the most influential features across all models. These are the features that consistently impact decision-making across various AI models in all our network intrusion datasets. Table 4 lists the top eight features for each dataset. For instance, features such as TCP_WIN_SCALE_IN, Destination Port, and dst_host_same_srv_rate are key influencers in the decision processes of all AI models for the RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD datasets, respectively. In addition, in the RoEduNet-SIMARGL2021 dataset, features related to TCP window settings, such as TCP_WIN_MSS_IN and TCP_WIN_SCALE_IN, are highlighted as significant. This indicates that TCP window configurations play a crucial role in assessing network behavior and detecting anomalies. In the CICIDS-2017 dataset, the "Destination Port" and packet metrics similar to "Init_Win_Bytes_Backward" are emphasized, pointing to the importance of monitoring port activities and traffic patterns for intrusion detection. Similarly, for the NSL-KDD dataset, features focusing on service consistency across hosts, like "dst_host_same_srv_rate" and "dst_host_srv_count", are pivotal.

This consolidated feature ranking serves as a valuable resource for security analysts, enabling them to monitor these features in network traffic even when the common attack type is not known beforehand. Detailed explanations of these common top intrusion features are provided in Appendix A.

Feature Importance for Each Attack Type: We proceed to analyze the intrusion-specific feature importance, focusing on the top five features associated with each attack type. Tables 10 and 11 present these lists for the RoEduNet-SIMARGL2021 and CICIDS-2017 datasets, respectively. In our observations, we note that, across both datasets, certain features emerge consistently across different attack types. For instance, features like TCP_WIN_SCALE_IN and FLOW_DURATION_MS for RoEduNet-SIMARGL2021 and Destination Port and Init_Win_Bytes_Bwd for CICIDS-2017 are recurrently prominent. For the NSL-KDD dataset, as depicted in Table 12, the feature *dst_host_same_src_port_rate* consistently ranks high across various attack types, such as Probe, R2L, and U2R, underlining its significance in diverse intrusion scenarios. Similarly, features like *hot* and *dst_host_count* appear prominently across multiple attack categories.

These findings provide crucial insights for analysts when investigating specific network intrusions, guiding them to focus on particular parameters associated with each attack type. Moreover, this knowledge can inform the development of attack-specific solutions for intrusion detection systems (IDSs), leveraging feature awareness for different network intrusion types.

Table 10. The feature importance, indicated by the top five features for each attack type for the RoEduNet-SIMARGL2021 dataset, shows *TCP_WIN_SCALE_IN* as the predominant feature across all attack types.

Feature Rank	Normal	DoS	Port Scan
1	TCP_WIN_SCALE_IN	TCP_WIN_SCALE_IN	TCP_WIN_SCALE_IN
2	TCP_WIN_MAX_OUT	TCP_WIN_MIN_IN	FLOW_DURATION_MS
3	FLOW_DURATION_MS	TCP_WIN_MAX_IN	TCP_WIN_MAX_IN
4	TCP_WIN_MIN_OUT	FLOW_DURATION_MS	TCP_WIN_MAX_OUT
5	TCP_WIN_MAX_IN	TCP_FLAGS	TCP_WIN_MIN_IN

Table 11. The feature importance (given by the top five features) for each attack type for the CICIDS-2017 dataset. The feature *Destination Port* is the common top feature across the different attack types.

Feature Rank	Normal	DoS	Port Scan	Bot
1	Destination Port	Destination Port	Packet Length Mean	Destination Port
2	Bwd Packet Length Mean	Packet Length Std	Init_Win_Bytes_Fwd	Init_Win_Bytes_Bwd
3	Total Length of Bwd Packets	Init_Win_Bytes_Bwd	Packet Length Std	Init_Win_Bytes_Fwd
4	Packet Length Std	Total Length of Bwd Packets	Total Length Fwd Packets	Packet Length Std
5	Init_Win_Bytes_Fwd	Bwd Packet Length Max	Average Packet Size	Packet Length Mean
Feature Rank	Web Attack	Brute Force	Infiltration	
1	Destination Port	Destination Port	Destination Port	
2	Init_Win_Bytes_Bwd	Init_Win_Bytes_Bwd	Init_Win_Bytes_Bwd	
3	Init_Win_bytes_Fwd	Init_Win_bytes_Fwd	Init_Win_bytes_Fwd	
4	Bwd Packet Length Mean	Total Length of Fwd Packets	Bwd Packet Length Mean	
5	Packed Length Std	Total Length of Bwd packets	Total Length of Fwd Packets	

Table 12. The feature importance (top five features) for each attack type for the NSL-KDD dataset. The features *dst_host_same_src_port_rate*, *hot*, and *dst_host_count* are common top features across different attack types.

Feature Rank	Normal	DoS	Probe
1	dst_host_srv_count	dst_host_serror_rate	dst_host_serror_rate
2	src_bytes	diff_srv_rate	dst_host_same_src_port_rate
3	dst_host_same_src_port_rate	flag_S0	dst_host_same_srv_rate
4	service_http	serror_rate	src_bytes
5	hot	same_srv_rate	dst_bytes
Feature Rank	R2L	U2R	
1	count	dst_host_count	
2	dst_host_same_src_port_rate	dst_host_srv_count	
3	dst_bytes	dst_host_same_src_port_rate	
4	hot	hot	
5	dst_host_count	count	

Common Features Among Datasets: We finally show the common intrusion features that we found in the three datasets. Table 13 shows such common features. Although the three datasets use different features to describe network traffic, we conducted manual scrutiny of each feature’s documentation description in [34,63,64] to identify equivalent features between datasets. Although the datasets have different characteristics, we took this extra step to show the common network traffic features among them. Such an analysis is important because, even if our framework is dataset-agnostic and flexible, this experiment shows that we can still recognize some network traffic patterns (features) among different testbeds for network intrusion detection. Thus, these network traffic features should be monitored by the security analyst managing the network.

Table 13. The list of common features between the three datasets.

Feature Importance	CICIDS-2017	RoEduNet-SIMARGL2021	NSL-KDD
1	Flow Duration	FLOW_DURATION_MS	duration
2	Fwd Header Length	IN_BYTES	src_bytes
3	Bwd Header Length	OUT_BYTES	dst_bytes
4	Total Fwd Packet	IN_PKTS	-
5	Total Bwd Packet	OUT_PKTS	-
6	Packet Length Min	MIN_IP_PKT_LEN	-
7	Packet Length Max	MAX_IP_PKT_LEN	-

Runtime Analysis: We now present our runtime analysis.

- (a) **AI Model Training Runtime:** For the CICIDS-2017 dataset, Table 14 shows that training the AI models on millions of samples takes, at most, a few minutes for most of the AI models considered in our work. It also shows that selecting all features for training would lead to a longer training time compared to only selecting the top 15 features (2.1X average increase across all AI models). It also shows that KNN and RF have the fastest training runtimes, respectively. On the other hand, LightGBM has the slowest training runtime for this dataset. For the RoEduNet-SIMARGL2021 dataset, Table 14 indicates that KNN and DNN exhibit the shortest training runtimes, whereas SVM demonstrates the longest training runtime. Similarly, for the NSL-KDD dataset, Table 14 reveals that KNN consistently has the fastest training times regardless of feature selection. In contrast, SVM has the most extended training duration, particularly for 15 features, which takes 6.48 min. On the other hand, LightGBM's training is relatively efficient, taking around 0.72 min for 15 features.

Table 14. Runtime (in minutes) for both the setups of 15 features and all features for the CICIDS-2017, NSL-KDD, and RoEduNet-SIMARGL2021 datasets.

Runtime	CICIDS-2017				NSL-KDD				RoEduNet-SIMARGL2021	
	Train (15 Feat.)	Pred (15 Feat.)	Train (All)	Pred (All)	Train (15 Feat.)	Pred (15 Feat.)	Train (All)	Pred (All)	Train (15 Feat.)	Pred (15 Feat.)
RF	0.19	0.02	0.32	0.02	0.73	0.01	0.89	0.01	11.57	0.69
ADA	1.92	0.15	5.05	0.17	0.2	0.01	0.33	0.03	10.91	1.47
DNN	0.58	0.11	1.04	0.29	1.28	0.01	1.36	0.01	5.99	5.66
SVM	0.20	0.01	0.75	0.03	6.48	0.02	7.78	0.02	828.51	3.40
KNN	0.09	1.68	0.14	2.71	0.01	0.59	0.01	0.60	1.24	24.84
MLP	0.81	0.11	1.44	0.11	3.14	0.01	3.82	0.01	7.85	1.68
LightGBM	7.58	0.02	13.35	0.11	0.72	0.001	0.78	0.001	9.01	0.04

- (b) **AI Classification Runtime:** In terms of classification (prediction), Table 14 shows that most of the AI models are fast, taking less than 0.5 min to complete the prediction task for the traffic test data for the CICIDS-2017 dataset, with RF being the AI model with the fastest prediction runtime. For the NSL-KDD dataset, the prediction times are similarly swift, with LightGBM showcasing an almost instantaneous prediction speed. However, KNN took notably longer, especially when considering all features. Table 14 indicates that the classification runtime for the RoEduNet-SIMARGL2021 dataset is longer compared to both the CICIDS-2017 and NSL-KDD datasets, attributable mainly to its size, which is roughly 15X that of the CICIDS-2017 dataset. Despite KNN's efficient training for RoEduNet-SIMARGL2021, it lagged in prediction speed. We emphasize that such times may vary with the usage of different computing resources, but intuitions about the ranking of different models' time performance would be consistent.
- (c) **Overhead Time for XAI Explanations:** We finally show that the gain from generating XAI explanations does not incur a large time overhead for most AI models. Table 15 shows how much time it takes to generate SHAP and LIME explanations. It shows that LIME is very efficient for all AI models (takes less than a minute on average)

to generate local explanations. For SHAP, the RF, DNN, and LightGBM models incur very little time to generate global explanations, irrespective of the number of samples for such explanations. On the other hand, the ADA and KNN models are not scalable to a larger number of samples. To test LIME globally, we tested generating LIME explanations for one thousand samples. Table 15 illustrates that LIME delivers consistent performance across all models but requires more time compared to SHAP for RF, DNN, and LightGBM. However, LIME performs better for the other models. In Table 15, several experiments were not completed due to the considerably longer time needed to complete them. It is worth noting, however, that the duration of these experiments still remained within the estimated time frame indicated by SHAP during the program's execution.

Table 15. The efficiency, measured in hours, in generating SHAP and LIME explanations across various AI models and differing sample sizes.

SHAP Efficiency (Samples × Hours)	RF	DNN	LGBM	MLP	ADA	SVM	KNN
100	0.001	0.001	0.001	0.014	0.129	0.041	1.104
500	0.002	0.004	0.001	0.925	3.557	0.841	28.33
2500	0.011	0.029	0.002	21.18	99.00	20.87	706.0
10,000	0.046	0.306	0.006	385.0	1512	343.0	1654
LIME Efficiency (Samples × Hours)	RF	DNN	LGBM	MLP	ADA	SVM	KNN
1000 (RoEduNet-SIMARGL2021)	1.401	1.452	1.423	1.466	1.416	1.483	6.666
1000 (CICIDS-2017)	3.812	3.766	3.750	3.855	3.956	3.816	15.816
1000 (NSL-KDD)	2.287	2.110	2.337	2.389	2.478	3.112	12.878
SHAP Local Efficiency (Minutes)	0.001	0.001	0.001	0.001	0.001	0.001	0.001
LIME Local Efficiency (Minutes)	0.351	0.311	0.294	0.284	0.294	0.306	0.357

4.4. Baseline Systems

We compare our feature extraction approach with two baseline systems [35,36]. In [36], the authors proposed a feature importance method that is based on the information gain of each feature based on entropy. On the other hand, the authors of [35] selected the feature importance by applying the K-best method [69]. Table 16 shows the summary of the top features extracted by these baselines.

Table 16. The list of top significant features according to baselines. References [35,36] were used for RoEduNet-SIMARGL2021 and CICIDS-2017, respectively. Ref. [34] was also used for the NSL-KDD dataset.

Feature Importance	CICIDS-2017	RoEduNet-SIMARGL2021	NSL-KDD
1	Packet Length Std	FLOW_DURATION_MILLISECONDS	same_srv_rate
2	Total length of Bwd Packets	FIRST_SWITCHED	flag_SF
3	Subflow Backward Bytes	TOTAL_FLOWS_EXP	flag_S0
4	Destination Port	TCP_WIN_MSS_IN	dst_host_srv_error_rate
5	Packet Length Variance	LAST_SWITCHED	dst_host_error_rate
6	Bwd Packet Length Mean	TCP_WIN_MAX_IN	error_rate
7	Avg Bwd Segment Size	TCP_WIN_MIN_IN	srv_error_rate
8	Init_Win_Bytes_Backward	TCP_WIN_MIN_OUT	dst_host_same_srv_rate

Top Features' Extraction: To understand the difference, we first compare our resulting features (Table 4) with the baseline features (Table 16). We observe that the order of feature importance determined by our method does not match those obtained by the baselines for both datasets. However, there are common features among both feature extraction methods. For example, TCP_WIN_MAX_IN, TCP_WIN_MIN_IN, TCP_WIN_MIN_OUT, and FLOW_DURATION_MILLISECONDS are common top features for the RoEduNet-SIMARGL2021 dataset.

For NSL-KDD, the overlapping top features between our method and the baselines are dst_host_srv_count, dst_host_same_src_port_rate, and count. We accentuate that none

of these baselines explain their feature selection using SHAP and that both of them used a lower number of AI models.

Effect of Different Top Features: We next compare the performance of AI models under the top five significant features selected by XAI-IDS versus those selected by information gain [36] and K-best [35]. Table 17 shows that XAI-IDS has superior performance (given by bold text) in eleven of fourteen AI models for the first two datasets (i.e., six of the seven models for CICIDS-2017 and five models (including two ties) for RoEduNet-SIMARGL2021). For the NSL-KDD dataset, as shown in Table 18, XAI-IDS consistently outperforms K-best in four out of seven AI models (RF, DNN, KNN, and LightGBM). Moreover, in ADA and MLP, there are tied metrics, but XAI-IDS remains dominant. The only model where K-best has an edge or ties significantly with XAI-IDS is SVM.

Table 17. A comparison of the models' performance metrics under top features selected by XAI-IDS versus those by information gain [36] and K-best [35]. XAI-IDS has superior performance (bold text) in eleven of fourteen AI models for the datasets.

CICIDS-2017								RoEduNet-SIMARGL2021							
AI Model	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc	AI Model	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc
RF (XAI-IDS)	0.99	0.96	0.96	0.96	0.98	0.97	0.98	RF (XAI-IDS)	0.99	0.99	0.99	0.99	0.99	0.99	0.99
RF (IG [36])	0.97	0.90	0.90	0.90	0.94	0.89	0.98	RF (K-best [35])	0.99	0.99	0.99	0.99	0.99	0.98	0.99
ADA (XAI-IDS)	0.93	0.78	0.78	0.78	0.87	0.74	0.95	ADA (XAI-IDS)	0.84	0.76	0.76	0.76	0.82	0.64	0.36
ADA (IG [36])	0.93	0.76	0.76	0.76	0.86	0.73	0.93	ADA (K-best [35])	0.69	0.53	0.53	0.53	0.65	0.30	0.48
DNN (XAI-IDS)	0.94	0.80	0.80	0.80	0.88	0.77	0.47	DNN (XAI-IDS)	0.99	0.99	0.99	0.99	0.99	0.98	0.99
DNN (IG [36])	0.89	0.64	0.64	0.64	0.79	0.58	0.53	DNN (K-best [35])	0.99	0.99	0.99	0.99	0.99	0.98	0.99
SVM (XAI-IDS)	0.99	0.97	0.97	0.97	0.90	0.97	0.66	SVM (XAI-IDS)	0.99	0.99	0.99	0.99	0.99	0.99	0.35
SVM (IG [36])	0.97	0.89	0.89	0.89	0.94	0.88	0.53	SVM (K-best [35])	0.43	0.15	0.15	0.15	0.36	−0.27	0.40
KNN (XAI-IDS)	0.99	0.99	0.99	0.99	0.99	0.99	0.89	KNN (XAI-IDS)	0.75	0.63	0.63	0.63	0.72	0.45	0.59
KNN (IG [36])	0.97	0.90	0.90	0.90	0.94	0.89	0.88	KNN (K-best [35])	0.75	0.63	0.63	0.63	0.72	0.45	0.59
MLP (XAI-IDS)	0.96	0.88	0.88	0.88	0.93	0.86	0.98	MLP (XAI-IDS)	0.76	0.64	0.64	0.64	0.73	0.46	0.46
MLP (IG [36])	0.96	0.87	0.87	0.87	0.93	0.85	0.96	MLP (K-best [35])	0.76	0.64	0.64	0.64	0.73	0.46	0.60
LightGBM (XAI-IDS)	0.97	0.92	0.92	0.92	0.95	0.90	0.56	LightGBM (XAI-IDS)	0.55	0.33	0.33	0.33	0.49	0.00	0.69
LightGBM (IG [36])	0.97	0.92	0.92	0.92	0.95	0.91	0.38	LightGBM (K-best [35])	0.87	0.81	0.81	0.81	0.86	0.71	0.96

Table 18. A comparison of the models' performance metrics under top features selected by XAI-IDS versus those by K-best [35]. XAI-IDS has superior performance (bold text) in six of seven AI models for the NSL-KDD dataset.

AI Model	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc
RF (XAI-IDS)	0.84	0.78	0.42	0.36	0.68	0.32	0.62
RF (K-best [35])	0.82	0.40	0.35	0.33	0.60	0.24	0.60
ADA (XAI-IDS)	0.81	0.41	0.47	0.21	0.63	0.27	0.58
ADA (K-best [35])	0.81	0.34	0.39	0.29	0.60	0.24	0.62
DNN (XAI-IDS)	0.88	0.51	0.48	0.46	0.69	0.40	0.69
DNN (K-best [35])	0.87	0.56	0.44	0.58	0.68	0.56	0.62
SVM (XAI-IDS)	0.87	0.60	0.43	0.43	0.66	0.36	0.70
SVM (K-best [35])	0.88	0.48	0.44	0.44	0.67	0.37	0.70
KNN (XAI-IDS)	0.88	0.52	0.54	0.73	0.72	0.47	0.73
KNN (K-best [35])	0.88	0.52	0.45	0.50	0.67	0.50	0.67
MLP (XAI-IDS)	0.90	0.75	0.54	0.72	0.51	0.33	0.68
MLP (K-best [35])	0.87	0.44	0.43	0.51	0.71	0.39	0.70
LightGBM (XAI-IDS)	0.84	0.39	0.41	0.37	0.68	0.26	0.64
LightGBM (K-best [35])	0.80	0.33	0.40	0.35	0.66	0.26	0.62

Effect of XAI-Based Feature Selection on Precision and Recall: We interpret the results of our metrics when comparing them with the baselines as follows. We compare these metrics under our top features against those selected by the baselines (K-best and information gain). Tables 17 and 18 show the results. In particular, when analyzing

these tables, our main findings are as follows. For CICIDS-2017, our framework has the same or better precision and recall in six of the seven models. Our framework also has the same or better performance for all seven AI models for all other metrics except the Matthews correlation coefficient for LightGBM. For the RoEduNet-SIMARGL2021 dataset, our framework has better precision and recall for all models except LightGBM. The same findings were found for all metrics except the AUCROC for ADA, SVM, and MLP. For the NSL-KDD dataset, Table 18 shows that our framework has better precision for all models except the DNN, while it has better recall in all models except SVM. To summarize, we see that our framework is better in almost all models analyzed for the three datasets, except for LightGBM for the RoEduNet-SIMARGL2021 dataset and SVM for the NSL-KDD dataset.

Finally, we believe that this experiment is important since depending on a lower number of features is important for the better explainability of AI models. In other words, explaining the decision-making of AI models using the top 5 features can be used in reality instead of explaining it using the top 20 features (which would be impractical and time-consuming for security analysts).

4.5. Micro-Experiments

Effect of Feature Normalization: Table 19 illustrates the impact of feature normalization on the DNN, SVM, and RF across the CICIDS-2017, RoEduNet-SIMARGL2021, and NSL-KDD datasets. Notably, we observe that feature normalization consistently leads to improved accuracy across all three datasets.

Upon examining the results presented in Table 19, we find that feature normalization generally enhances accuracy for the DNN, SVM, and RF models employed in this micro-experiment (e.g., an increase in the accuracy of the DNN model on the RoEduNet-SIMARGL2021 dataset from 0.62 to 0.99) across the various datasets under consideration. It is worth mentioning that this trend holds true for the majority of cases (46 out of 54 cases), indicating a notable improvement in model performance facilitated by feature normalization. However, there are a few exceptions (8 cases out of 54) where normalization does not yield enhanced performance, which is particularly evident for the NSL-KDD dataset.

Given these findings, we advocate for feature normalization across all our AI model results within the scope of *our framework*, as discussed in the paper.

Table 19. The effect of feature normalization on the performance of AI models for the three datasets in XAI-IDS.

AI Model—DNN	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc
CICIDS-2017 (No Norm.)	0.94	0.80	0.80	0.80	0.88	0.77	0.80
CICIDS-2017 (Norm)	0.95	0.83	0.83	0.83	0.90	0.80	0.96
RoEduNet-SIMARGL2021 (No Norm.)	0.62	0.24	0.24	0.24	0.43	0.13	0.50
RoEduNet-SIMARGL2021 (Norm)	0.99	0.99	0.99	0.99	0.99	0.98	0.99
NSL-KDD (No Norm.)	0.76	0.47	0.45	0.62	0.70	0.57	0.55
NSL-KDD (Norm)	0.88	0.51	0.48	0.46	0.69	0.40	0.69
AI Model—SVM	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc
CICIDS-2017 (No Norm.)	0.98	0.92	0.92	0.92	0.95	0.90	0.50
CICIDS-2017 (Norm)	0.98	0.92	0.92	0.92	0.95	0.90	0.60
RoEduNet-SIMARGL2021 (No Norm.)	0.47	0.21	0.21	0.21	0.41	−0.18	0.50
RoEduNet-SIMARGL2021 (Norm)	0.75	0.62	0.62	0.62	0.72	0.44	0.49
NSL-KDD (No Norm.)	0.75	0.54	0.98	0.70	0.65	0.46	0.67
NSL-KDD (Norm)	0.87	0.60	0.43	0.43	0.66	0.36	0.70
AI Model—RF	Acc	Prec	Rec	F1	Bacc	Mcc	AucRoc
CICIDS-2017 (No Norm.)	0.99	0.99	0.99	0.99	0.99	0.99	0.56
CICIDS-2017 (Norm)	0.99	0.99	0.99	0.99	0.99	0.99	0.61
RoEduNet-SIMARGL2021 (No Norm.)	0.66	0.29	0.29	0.29	0.47	0.06	0.50
RoEduNet-SIMARGL2021 (Norm)	0.98	0.95	0.95	0.95	0.97	0.95	0.51
NSL-KDD (No Norm.)	0.80	0.75	0.38	0.67	0.67	0.67	0.60
NSL-KDD (Norm)	0.84	0.78	0.42	0.36	0.68	0.32	0.62

5. Related Work

This section provides an overview of existing efforts in leveraging explainable artificial intelligence (XAI) for intrusion detection systems (IDSs), highlighting the distinctions between our work and previous research, as summarized in Table 20.

Overview of Prior Works: The surveys conducted by [70,71] offer a comprehensive examination of ongoing endeavors aimed at developing explainable AI-based intrusion detection systems (XAI-IDS), accompanied by a taxonomy for standardizing concepts. This survey delineates between black-box models (opaque models) and white-box models (transparent models), analyzing their trade-offs in terms of explainability and performance.

Several recent studies have explored the application of XAI techniques in IDSs [72–75]. For example, ref. [72] utilized decision tree algorithms to extract rules in an IDS using the KDD dataset. In another work, ref. [73] incorporated LIME to provide local explanations for an SVM model in an IDS. Similarly, ref. [74] integrated the CIA principle to enhance the explainability of AI models using the CICIDS-2017 dataset.

In addition, the landscape of intrusion detection in IoT and cloud computing is replete with diverse challenges and innovative solutions. The work presented in [29] discusses the deployment of bio-inspired algorithms for feature selection, significantly enhancing the predictive accuracy of the IDS; however, they highlight the inherent complexities and computational demands that these techniques introduce. The work in [76] surveys several XAI techniques, such as SHAP and LIME, and their applications and challenges in healthcare, such as the complexity of medical data and the critical need for accurate interpretations. Furthermore, the IDS frameworks in [77,78] illustrate successful adaptation in applying an IDS and XAI to the autonomous vehicle area. They emphasize the need for XAI to provide clarity on AI decisions, enhancing security measures against cyber threats. On the other hand, the works in [29,31,32] propose frameworks to enhance IDSs in the context of IoT and SCADA Systems. It is also worth mentioning [29,30,79], which do the same for network intrusion detection, leveraging ensemble learning and neural network applications. The authors of [80] focused on identifying DNS over HTTPS (DoH), a protocol that encrypts DNS queries to enhance privacy but poses challenges for network administrators in detecting malicious activities. It also applies SHAP to explain its results. These referenced works show the advancements and obstacles in the field, thus setting the stage for addressing these gaps through the proposed framework.

Comparison with Our Work: While previous efforts in XAI for IDSs are commendable, our research distinguishes itself in several key aspects. Unlike existing studies, which often focus on individual XAI techniques or specific datasets, our project introduces an integrated end-to-end XAI framework explicitly crafted for network intrusion detection. We perform detailed evaluations of multiple black-box AI models, examine the effects of feature selection, and offer comprehensive insights into the importance of features for different attack types. Additionally, we present a comparative analysis of various XAI methodologies, such as SHAP and LIME, to demonstrate their effectiveness in explaining the decision-making processes of AI models used in IDSs.

This research introduces a novel approach to intrusion detection that is distinctly advanced beyond previous contributions and addresses the limitations observed in the referenced studies. Unlike the methodologies presented in [29–32,76–80], which primarily focus on leveraging existing models and algorithms for feature selection and detection or on XAI but applied to different scenarios, such as IoT, autonomous vehicles, or healthcare, this study integrates these techniques within a unique framework that employs explainable AI (XAI) to enhance transparency and interpretability in intrusion detection systems in the context of network intrusion detection. The integration of XAI not only builds on the robust detection capabilities highlighted in [71,78] but also provides the benefits of interpretability and trust [70,71]. Furthermore, our work extends beyond mere model performance evaluation by investigating the interpretability of AI models and their feature importance, facilitating a deeper understanding of the underlying mechanisms driving

intrusion detection. By addressing these research gaps and offering a comprehensive framework, our work contributes to advancing the field of XAI-enabled IDSs.

Table 20. A comparison between different aspects of our work and those of prior relevant works on XAI for network intrusion detection (including methods for explanations, diversity of datasets, XAI explainers, AI models, time analysis, open-sourcing, and multiclass classification).

Paper	Diversity of Datasets	Diversity of AI Models	Multiple XAI Explainers	Attack-Specific Features
Our Work	Yes	Yes	Yes	Yes
DeepAID [53]	Yes	No	Yes	No
Kitsune [55]	Yes	Yes	No	No
OWAD [81]	Yes	Yes	Yes	No
Feature-oriented Design [54]	No	No	Yes	No
Explainable ML Framework [82]	No	Yes	No	No
An Evaluation of XAI in Security [37]	Yes	No	Yes	No
E-XAI: Evaluating XAI for IDS [83]	Yes	Yes	Yes	No
Paper	Time Analysis	Open Source	Model-Specific Features	Local and Global Scopes
Our Work	Yes	Yes	Yes	Yes
DeepAID [53]	Yes	Yes	No	Yes
Kitsune [55]	Yes	Yes	No	No
OWAD [81]	Yes	Yes	No	No
Feature-oriented Design [54]	No	No	Yes	No
Explainable ML Framework [82]	No	No	Yes	Yes
An Evaluation of XAI in Security [37]	Yes	Yes	Yes	No
E-XAI: Evaluating XAI for IDS [83]	Yes	Yes	No	No

Other research endeavors are also noteworthy for their innovative contributions. The study in [81] introduced OWAD (Open-World Anomaly Detection), a novel framework that innovatively addresses normal concept drift in deep learning-based models. In parallel, the study in [53] developed DeepAID, a new framework designed to overcome the challenges faced by unsupervised deep learning-based anomaly detection systems. Focusing on camera network systems, the research [55] presented Kitsune, an online (i.e., real-time) anomaly detection system tailored for network intrusion detection. Table 20 outlines the key distinctions between our work and these previous studies.

Feature Selection for Intrusion Detection: Several works investigate effective methods for feature selection [35,36,84,85]. In particular, most of these methods focus solely on dataset analysis using information gain [36,84,85], correlation-based methods [86], or the K-best method [35,87]. However, new research is shifting this notion by proposing frameworks that incorporate an AI model as well [82]. In a slightly different approach, the work in [54] built a dashboard for intrusion detection using JavaScript and D3 that caters to the requirements of field experts. It showcases a panel with explanations of a given sample using LIME, Wordle Saliency, and Bar Saliency Chart. References [88,89] demonstrate the creation of LEMNA, an explainer that uses backpropagation and the Lasso technique specifically designed for IDSs. CADE [89] is designed to identify and handle drift that can occur in the face of a new attack (i.e., an attack that the AI model has never seen).

Evaluation of XAI Metrics: It is crucial to delineate the differences between this work and our earlier study [83]. Previously, our research concentrated solely on assessing the explainable AI (XAI) methods SHAP and LIME by employing six distinct XAI metrics: Descriptive Accuracy, Sparsity, Robustness, Efficiency, Stability, and Completeness, as detailed in [37]. In contrast, the current study introduces a proof-of-concept framework

aimed at aiding security analysts by offering explanations for the analyzed samples using LIME and SHAP (refer to Figure A5). Additionally, this paper investigates the effectiveness of various AI models utilizing different feature selection methods derived from SHAP global explanations. We specifically extract model-specific and intrusion-specific features and assess these feature selection strategies against prior baseline methods.

By broadening our focus from merely evaluating XAI techniques to implementing these methods within an IDS framework, we strive to provide an extensive understanding of how XAI can enhance both the interpretability and performance of IDSs.

6. Discussion

6.1. Challenges in Using XAI Methods for Real-World IDSs

It is crucial to recognize the challenges inherent in developing interpretation methods for network intrusion detection systems (IDSs), which present a stark contrast to other fields like text analysis and computer vision, where explainable AI (XAI) methods are more widely utilized [37,82]. Although XAI techniques such as LIME and SHAP hold considerable promise, there is a pressing need for further developments to tailor these methods more effectively for network cybersecurity [90]. Nonetheless, the XAI-based analysis conducted in this study marks a significant advancement in enhancing the application of XAI within the domain of network intrusion detection.

Another black-box XAI tool that can be considered in this context is LEMNA [37]. While LEMNA offers specific benefits in handling complex nonlinear data relationships, it faces challenges in terms of user accessibility and adaptability compared to LIME and SHAP. Additionally, LEMNA lacks comprehensive testing similar to SHAP and LIME, which may impact its reliability and performance. Despite its potential value in aiding security analysts in understanding and mitigating the risks associated with deep learning-based systems, LEMNA, like software fuzzing methods, carries inherent risks [88]. These limitations underscore the intricacies involved in utilizing LEMNA in cybersecurity applications.

6.2. Importance of Our XAI Framework for IDSs

In the current era of rapid information growth, the frequency of network attacks is escalating, as highlighted by recent research from the Center for Strategic & International Studies (CSIS) [91]. Despite advances in intrusion detection systems (IDSs) over the years, security analysts face the daunting challenge of analyzing potential attacks that occur more rapidly than they can be processed. Accurate explainable AI (XAI) intrusion detection systems represent a crucial solution to this issue.

The framework presented in this paper builds upon previous signature-based research [54,70,82], employing a method to identify the most influential features applicable across various models. For each of the seven AI models evaluated in this study, the key features are determined using SHAP and further refined through frequency analysis to pinpoint the most recurrent features. This methodology is also used to ascertain the most critical features for each type of attack. Additionally, our framework offers comprehensible explanations of AI decision-making processes and the pertinent influential features.

This approach is applied across three distinct datasets: NSL-KDD, CICIDS-2017, and RoEduNet-SIMARGL2021. Our research also includes a cost analysis of SHAP and LIME based on the number of samples utilized, offering insights into which AI models might be more cost-effective. This end-to-end XAI-based strategy extends beyond traditional multiclass classification for IDSs by highlighting the importance of influential features and providing tailored insights into specific types of attacks.

6.3. Insights on Chosen Datasets

To rigorously evaluate our XAI framework, we chose the following datasets: NSL-KDD, CICIDS-2017, and RoEduNet-SIMARGL2021. These datasets were specifically chosen due to their varied and comprehensive representations of network intrusions, which range from DoS to Bot attacks. By dissecting each dataset, we highlighted inherent

challenges, such as imbalanced classes, varied attack signatures, and differing traffic patterns. The analysis results of the application of XAI not only underline the relevance of these datasets to real-world network environments but also showcase how our XAI framework effectively identifies and explains key features across such diverse conditions. This depth of analysis ensures that our framework's capabilities are tested against scenarios that closely mimic the complexities encountered in actual network security operations, especially when considering RoEduNet-SIMARGL2021 since CICIDS-2017 and NSL-KDD are considered benchmark datasets for network intrusion detection tasks.

In particular, the selection of these three datasets for evaluating our XAI framework was driven by their distinct characteristics and widespread recognition in the network security community. NSL-KDD is an improved version of the KDD'99 dataset and addresses some of the inherent problems of its predecessor. CICIDS-2017 is valued for its realistic network traffic scenarios, which include a wide array of modern attacks. This dataset mimics the state of current network environments more accurately, providing rich testing security technologies. Lastly, RoEduNet-SIMARGL2021 offers data from a real operational network, adding valuable insights into the complexities and nuances of real-world network traffic that laboratory-created datasets may not replicate.

6.4. Evaluating Our XAI Framework Using Other Benchmark Datasets and Human Evaluation

In this study, we provided a detailed examination of our XAI framework utilizing the RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD datasets. Looking ahead, a valuable direction for future research will be to assess the effectiveness of our XAI framework on additional network intrusion detection benchmark datasets. Potential datasets for exploration include UNSW-NB15 [92], CICIDS-2018 [93], CTU-13 [94], UMass [95], and various vulnerability-based datasets [96]. This expansion would help further validate and refine our framework, broadening its applicability and enhancing its utility in diverse cybersecurity environments.

Applying and testing our framework on additional datasets and testbeds for network IDSs would broaden its applicability and provide insights into its generalizability. Furthermore, exploring a human evaluation of the explainability provided by our XAI framework would further enhance its applicability.

In particular, the work by Hoffman et al. [97] offers guidelines and forms for evaluating XAI from a human perspective. This study advocates for incorporating feedback from security experts to validate and enhance the generated explanations. Using this study as a guideline for incorporating human evaluation could help quantify the level of explanation provided by our XAI framework and ensure its effectiveness in real-world scenarios. Thus, integrating human evaluation into our framework represents a fruitful direction for future research.

6.5. Combining Different XAI Frameworks

One other future extension can be combining the considered black-box XAI methods in this work (SHAP and LIME) with white-box XAI methods (such as IG [98] and LRP [70]).

6.6. Effect of Bias of Top Intrusion Features

In considering the overall important features, it is essential to recognize the potential for bias in some of them, which could affect the dataset generation process. For example, "Destination Port" has been flagged as a potential source of bias [99], where its inclusion in AI network intrusion detection models might lead to contamination effects. Similarly, it is reasonable to speculate that TCP_WIN_SCALE_IN could have a comparable contaminating influence on the RoEduNet-SIMARGL2021 dataset.

It is important to clarify, however, that our XAI explanation and feature importance assessments are not solely reliant on individual features. Although some features may exhibit bias, the aggregate impact of multiple features should help in counteracting such biases. Nonetheless, as a precaution, we could include a caveat within the XAI

explanation whenever a top feature identified is potentially biased, such as “Destination Port” or TCP_WIN_SCALE_IN. This addition would furnish users with further context and encourage a more critical assessment of the model’s decisions in light of possible biases.

6.7. Time Performance of XAI Methods

The goal of our time performance analysis in Section 4 is to show which AI models are more efficient when used with black-box XAI methods for IDSs. This will depend on the user’s requirements. Our results show that some AI models will be more time-efficient when used for detecting intrusions if the IDS requires real-time inquiries from XAI methods. In particular, we found that while LIME shows consistent efficiency across most AI models, it is less effective with KNN. On the other hand, SHAP works efficiently with LGBM, RF, and DNN but faces significant performance issues with MLP, ADA, and KNN. Thus, if our framework is applied to a real-world scenario, we will recommend efficient models for each XAI method (here, SHAP and LIME). Overall, SHAP is more time-efficient for global explanations, whereas LIME is better in local explanation scenarios across all tested AI models (see Table 15).

7. Conclusions

This paper highlights the essential need for clear explanations of AI methods in intrusion detection by introducing an XAI framework specifically designed for network intrusion detection tasks. This framework provides both global and local explanations for AI models, identifying important model-specific and intrusion-specific features across various AI models and types of network intrusions. This research involves benchmarking seven black-box AI models on three real-world network intrusion datasets: RoEduNet-SIMARGL2021, NSL-KDD, and CICIDS-2017. Initially, it assesses their performance, and then it evaluates the XAI methods, uncovering insights into the overall importance of features, their contributions to model decisions, top features for each type of intrusion, and shared features among different models. Notably, the study shows that the use of XAI methods results in negligible additional time overhead. Additionally, the paper conducts a benchmark comparison with two previous solutions for feature selection in network intrusion detection, emphasizing the advantages of the features selected by the XAI framework. It also makes the source codes available, establishing a baseline XAI framework for network intrusion detection tasks, which encourages collaboration and further exploration in this area. However, the paper acknowledges the potential for future enhancements, particularly in consolidating all models and datasets into a more user-friendly repository.

Author Contributions: Conceptualization: O.A., T.G. and M.A.; data curation: O.A. and T.G.; formal analysis: O.A., T.G. and M.A.; funding acquisition: M.A.; investigation: O.A. and T.G.; methodology: O.A., T.G. and M.A.; project administration: M.A.; resources: T.G. and M.A.; software: O.A. and T.G.; supervision: M.A.; validation: M.A.; visualization: M.A.; writing—original draft: O.A. and T.G.; writing—review and editing: O.A., T.G. and M.A. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by Lilly Endowment (AnalytixIN). It was also supported by the Enhanced Mentoring Program with Opportunities for Ways to Excel in Research (EMPOWER); in part by the 1st Year Research Immersion Program (1RIP) Grants from the Office of the Vice Chancellor for Research at Indiana University–Purdue University Indianapolis; and in part by Lilly Endowment, Inc., through its support for the Indiana University Pervasive Technology Institute.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The authors share the source codes along with this submission. The URL for our database and codes is https://github.com/ogarreche/XAI_NIDS (accessed on 9 April 2024).

Acknowledgments: We thank Xiao Luo and Tianyi Li for their valuable feedback and suggestions on the work.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Explanations of Top Intrusion Features in XAI-IDS

Appendix A.1. Detailed Explainability of Top Intrusion Features in RoEduNet-SIMARGL2021 Dataset

We can explain the sample analyzed by LIME using the top five features. We first give detailed definitions of each of these features below.

Explaining Top Features from a Security Point of View:

- (1) **TCP_WIN_SCALE_IN:** This feature is relevant because certain port scanning techniques involve manipulating TCP window scaling factors to gather information about open ports or available services on a target network. A high or unusual value of TCP_WIN_SCALE_IN could indicate such activity.
- (2) **TCP_FLAGS:** The TCP flags can provide insights into the nature of network traffic. In the case of PortScan or Network Service Discovery, specific flag combinations (such as SYN-only packets) are commonly used to scan for open ports. The presence of specific TCP_FLAGS patterns can indicate port scanning behavior.
- (3) **TCP_WIN_MIN_OUT:** The minimum window size advertised by the sender (source) can provide information about the behavior of the scanning activity. Port scanners may use specific window sizes to optimize their scanning, and unusual values of TCP_WIN_MIN_OUT suggest scanning behavior.
- (4) **TCP_WIN_MSS_IN:** The Maximum Segment Size (MSS) used by the receiver in a TCP connection can offer insights into the scanning activity. Port scanners may manipulate the MSS value to gather information about the target network's characteristics. Unusual or unexpected values of TCP_WIN_MSS_IN could indicate scanning behavior.
- (5) **TOTAL_FLOW_EXP:** This refers to the number of flows exported so far. A high number of TOTAL_FLOW_EXP can be used as an explanation feature to indicate DoS attacks in which the attacker floods the network with many packets. It can also be used as an error troubleshooting tool to catch wrong biases in the AI model.

Appendix A.2. Detailed Explainability of Top Intrusion Features in NSL-KDD

Explaining Top Features from a Security Point of View:

- (1) **DST_HOST_SAME_SRV_RATE:** This feature is relevant because, during a network attack such as a DoS attack or port scanning, the attacker might repeatedly connect to the same service on a particular host. A high DST_HOST_SAME_SRV_RATE value may indicate such repetitive access to the same service, which could be a sign of suspicious activity.
- (2) **DST_HOST_SRV_COUNT:** This feature is important because it can highlight unusually high numbers of connections to a particular service, which could be indicative of a Brute-Force attack or a DoS attack. A high DST_HOST_SRV_COUNT may suggest that an attacker is repeatedly connecting to a service, attempting to gain unauthorized access or overload the service.
- (3) **DST_HOST_SAME_SRC_PORT_RATE:** This feature is relevant because, during certain network attacks, such as port scanning or DoS attacks, an attacker might make multiple connections from the same source port. A high DST_HOST_SAME_SRC_PORT_RATE could indicate such behavior, suggesting potential malicious activity.
- (4) **LOGGED_IN:** This feature is important because if an unexpected or unauthorized host is logged into a system, it can be an indication of a security breach. The feature could capture scenarios where an attacker has successfully gained unauthorized access to a system.

- (5) **DST_HOST_SERROR_RATE**: This feature is significant, as it could indicate the presence of a network attack. SYN errors may occur when there is an attempted connection to a closed port, which could be indicative of port scanning or a failed unauthorized access attempt. A high DST_HOST_SERROR_RATE may thus indicate malicious network behavior.

Appendix B. Local XAI Illustration on CICIDS-2017 Dataset

Figures A1 and A2 represent local explanations for SHAP and LIME, respectively, for the same RF prediction for a normal traffic sample from the CICIDS-2017 dataset. Figures A3 and A4 showcase the local explanations for the same methods applied to a sample from the NSL-KDD dataset. Such figures show different local explanations for the same instance. We see that most of the main (top) features remain the same for both explanations. However, LIME offers more details for a single instance than SHAP. First, there is a confidence score in LIME that shows how much the model believes that a class is correct, whereas in SHAP, we do not see that. Moreover, SHAP does not show the confidence score for other classes either (seen in LIME). On the other hand, the SHAP's explanation (waterfall plot) is easier to read and interpret from a visualization point of view, where it clearly shows the contribution of each feature to the AI model. We believe that, although both methods are different, the combination of the two can serve as a cross-validation for the local explanation of the AI model's prediction.

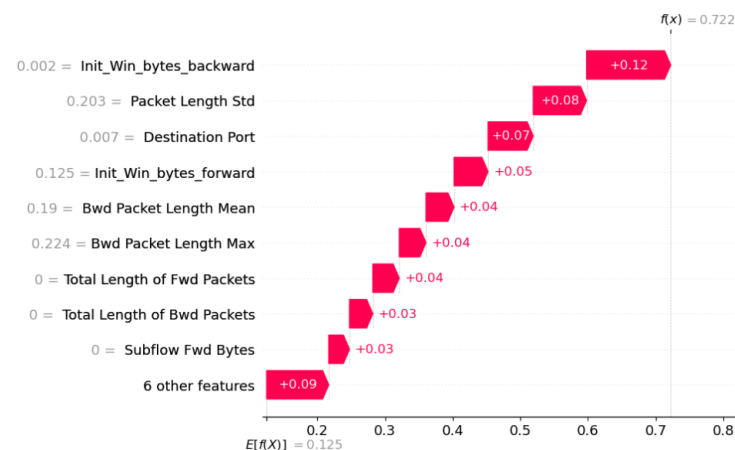


Figure A1. An example of a local explanation for correctly predicting a normal traffic sample by the RF model using SHAP for the CICIDS-2017 dataset.

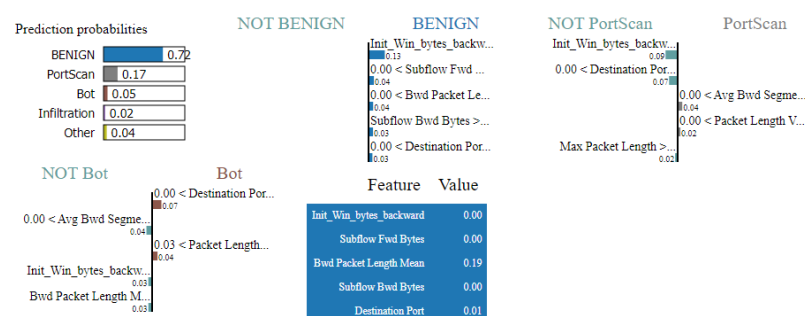


Figure A2. An example of a local explanation for correctly predicting a normal traffic sample by the RF model using LIME for the CICIDS-2017 dataset.

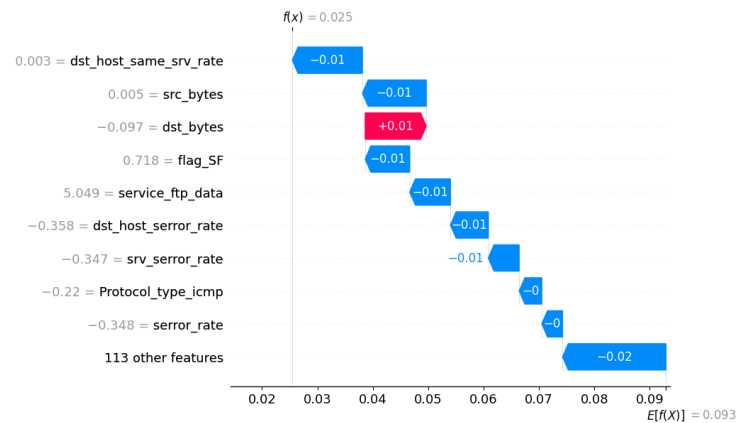


Figure A3. An example of a local explanation for correctly predicting a normal traffic sample by the RF model using SHAP for the NSL-KDD dataset.

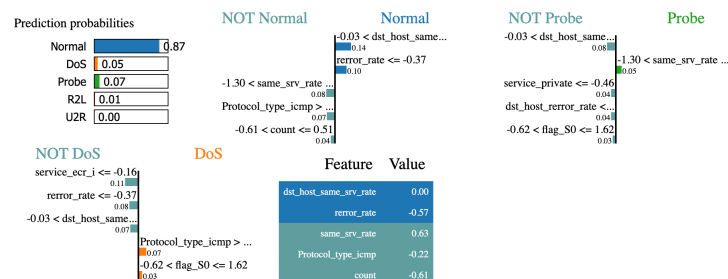


Figure A4. An example of a local explanation for correctly predicting a normal traffic sample by the RF model using LIME for the NSL–KDD dataset.

Appendix C. AI Models and Hyperparameters

Table A1. Classifier details and hyperparameters.

Classifier	Hyperparameters
Deep neural network (DNN)	Architecture: Input layer with ReLU activation function, dropout layer with dropout rate of 0.01, hidden layer with 16 neurons and ReLU activation function, and output layer with softmax activation function. Loss function: Categorical cross-entropy. Optimization algorithm: Adaptive momentum (ADAM). Training: 11 epochs with batch size of 1024.
Random forest (RF)	n_estimators: 100; maximum tree depth: 10; minimum number of samples required to separate internal node: 2 (other parameters set to default).
AdaBoost (ADA)	Maximum number of estimators: 50; weight applied to each classifier: 1; base estimator: Decision Tree Classifier.
K-nearest neighbor (KNN)	Number of neighbors: 5; weights: uniform; search algorithm: auto (default parameters).
Support vector machine (SVM)	Kernel: Linear; gamma: 0.5; probability: True; regularization: 0.5.
Multi-layer Perceptron (MLP)	Same setup as DNN.
LightGBM	n_splits: 10; n_repeats: 3; error_score: raise; n_jobs: 1; scoring: accuracy (default parameters).

Appendix D. Detailed Illustrations of Explanations of AI Decision-Making in XAI-IDS

We now provide more examples showing illustrations of explanations of AI decision-making in XAI-IDS. Figure A5 shows such illustrations.

Local XAI Explanations on NSL-KDD Dataset: Figure A5 represents a local explanation for LIME for a specific RF prediction from the NSL-KDD dataset. This figure shows a clear local explanation for a misclassified instance. The local explanation for LIME provides a confidence score that shows the model's belief in the correctness of a class,

adding further detail to our understanding of the model’s decision-making process. In the particular case of the misclassification shown here, the model incorrectly classified a Probe attack as Normal due to the influence of several intrusion features.

For instance, the negative impact of “service_ntp_u” and “num_root” suggested normal behavior, which can be attributed to fewer operations as root and less usage of NTP service. Conversely, the presence of “service_IRC” indicated suspicious activity, hinting at a Probe attack. Features like the low values of “service_domain” and “service_tftp_u” further contributed to the Normal classification, associating these services with regular network activities. Simultaneously, the model picked up on suspicious behavior from features such as the high number of “su_root” commands attempted (“su_attempted”) and the presence of “service_pm_dump”. These could be indicative of potential exfiltration attempts, characteristic of a Probe attack. However, the combined impact of these features was insufficient to correct the classification. This analysis underscores the need for the model to be retrained to weigh these features correctly. We believe that this detailed examination of the model’s local decision-making process, provided by our XAI framework, can significantly enhance our understanding and ability to improve black-box AI models.

Appendix E. Extended XAI Results

Here, we show the extended results of the XAI techniques for the AI models. In particular, Figures A7 and A8 show the beeswarm plots for our different AI models for the RoEduNet-SIMARGL2021 and CICIDS-2017 datasets, respectively.

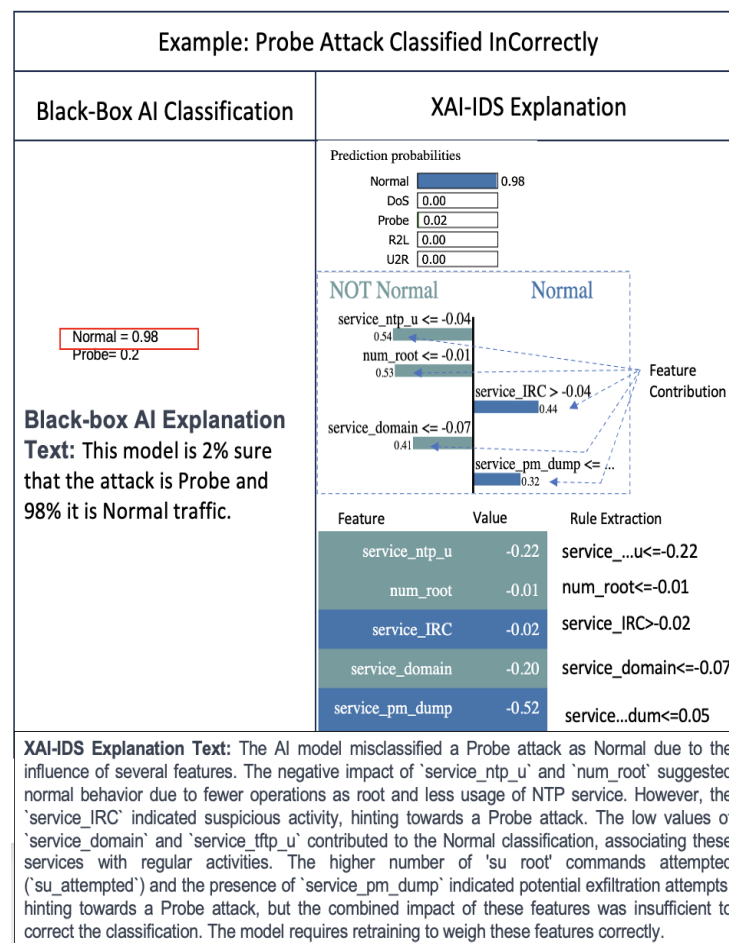
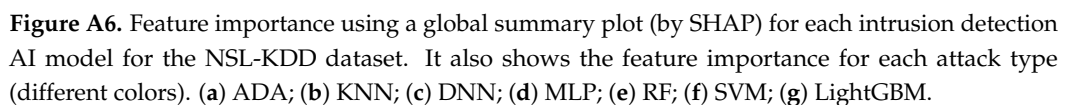
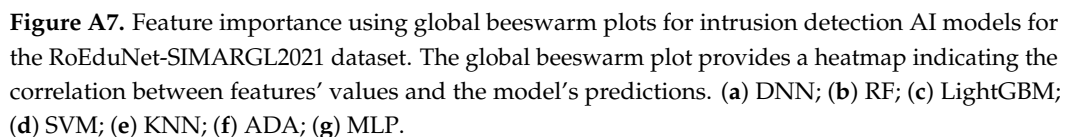


Figure A5. An example of the enhancement of the explanation of the AI model’s decision-making using our XAI framework versus a black-box AI model. The example shows a wrong classification by the AI model on the NSL–KDD dataset.





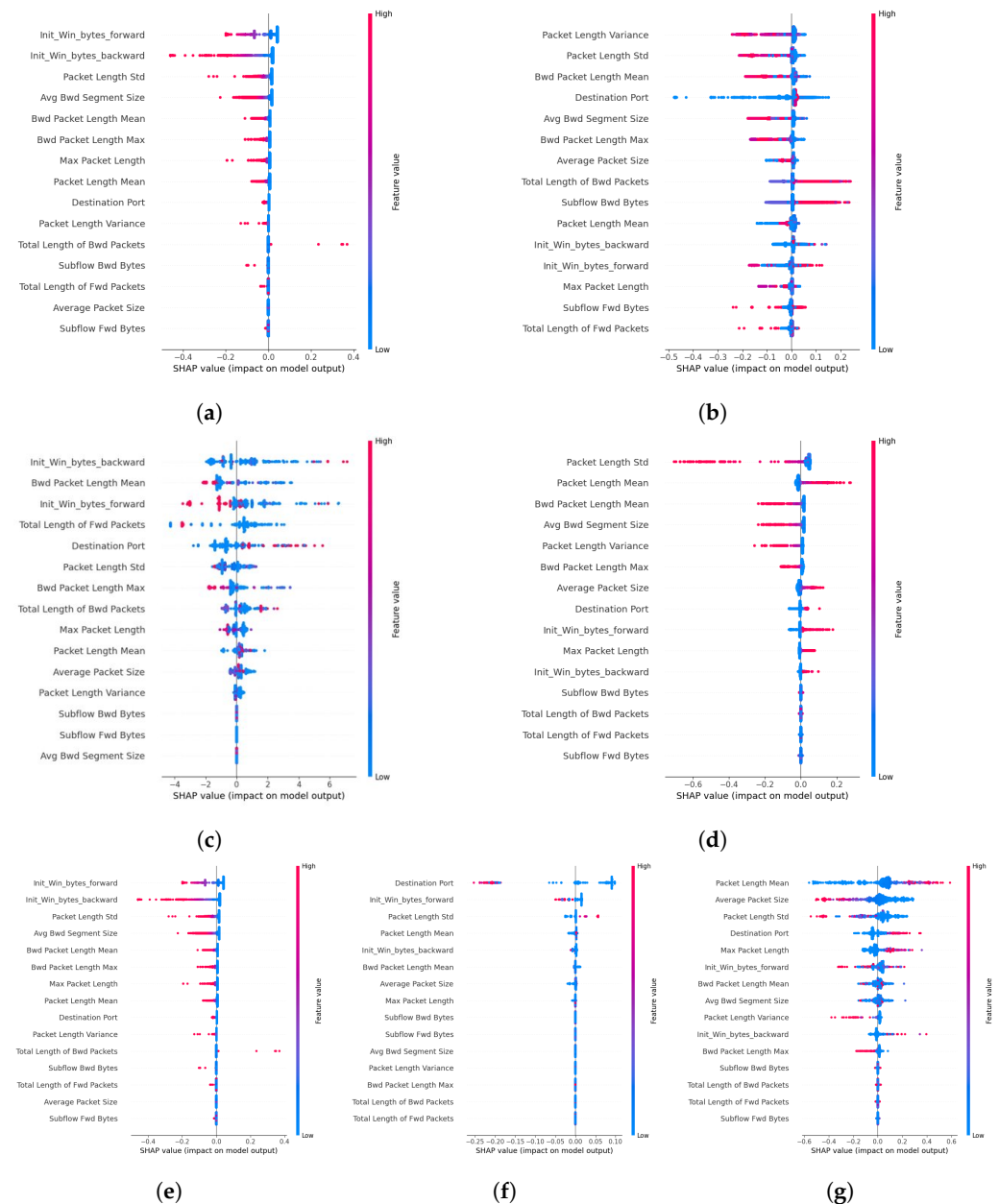


Figure A8. Feature importance using global beeswarm plots for intrusion detection AI models for the CICIDS-2017 dataset. The beeswarm plot is a more intricate visualization tool compared to the summary plot, as it depicts not only the significance of features but also their associations with the prediction (with a heatmap for correlation between feature values and models). (a) DNN; (b) RF; (c) LightGBM; (d) SVM; (e) KNN; (f) ADA; (g) MLP.

References

1. Northcutt, S.; Novak, J. *Network Intrusion Detection*; Sams Publishing: Clay Township, IN, USA, 2002.
2. Vasiliadis, G.; Antonatos, S.; Polychronakis, M.; Markatos, E.P.; Ioannidis, S. Gnort: High performance network intrusion detection using graphics processors. In Proceedings of the Recent Advances in Intrusion Detection: 11th International Symposium, RAID 2008, Cambridge, MA, USA, 15–17 September 2008; Proceedings 11; Springer: Berlin/Heidelberg, Germany, 2008; pp. 116–134.
3. Apruzzese, G.; Andreolini, M.; Ferretti, L.; Marchetti, M.; Colajanni, M. Modeling realistic adversarial attacks against network intrusion detection systems. *Digit. Threat. Res. Pract. (DTRAP)* **2022**, *3*, 1–19. [\[CrossRef\]](#)
4. Wolsing, K.; Wagner, E.; Saillard, A.; Henze, M. IPAL: Breaking up silos of protocol-dependent and domain-specific industrial intrusion detection systems. In Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses, Limassol, Cyprus, 26–28 October 2022; pp. 510–525.
5. Patcha, A.; Park, J.M. An overview of anomaly detection techniques: Existing solutions and latest technological trends. *Comput. Netw.* **2007**, *51*, 3448–3470. [\[CrossRef\]](#)

6. Asad, H.; Gashi, I. Dynamical analysis of diversity in rule-based open source network intrusion detection systems. *Empir. Softw. Eng.* **2022**, *27*, 4. [\[CrossRef\]](#)
7. Ilgun, K.; Kemmerer, R.A.; Porras, P.A. State transition analysis: A rule-based intrusion detection approach. *IEEE Trans. Softw. Eng.* **1995**, *21*, 181–199. [\[CrossRef\]](#)
8. Li, L.; Yang, D.Z.; Shen, F.C. A novel rule-based Intrusion Detection System using data mining. In Proceedings of the 2010 3rd International Conference on Computer Science and Information Technology, Chengdu, China, 9–11 July 2010; IEEE: New York, NY, USA, 2010; Volume 6, pp. 169–172.
9. Snapp, S.R.; Smaha, S.E.; Teal, D.M.; Grance, T. The {DIDS}(Distributed Intrusion Detection System) Prototype. In Proceedings of the USENIX Summer 1992 Technical Conference (USENIX Summer 1992 Technical Conference), San Antonio, TX, USA, 8–12 June 1992.
10. Jackson, K.A.; DuBois, D.H.; Stallings, C.A. *An Expert System Application for Network Intrusion Detection*; Technical report; Los Alamos National Lab. (LANL): Los Alamos, NM, USA, 1991.
11. Muhammad, M.; Saleem, A. Intelligent Intrusion Detection System for Apache Web Server Empowered with Machine Learning Approaches. *Int. J. Comput. Innov. Sci.* **2022**, *1*, 21–28.
12. Tabassum, A.; Erbad, A.; Guizani, M. A survey on recent approaches in intrusion detection system in iots. In Proceedings of the 2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC), Tangier, Morocco, 24–28 June 2019; IEEE: New York, NY, USA, 2019; pp. 1190–1197.
13. Buczak, A.L.; Guven, E. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Commun. Surv. Tutor.* **2015**, *18*, 1153–1176. [\[CrossRef\]](#)
14. Dina, A.S.; Manivannan, D. Intrusion detection based on Machine Learning techniques in computer networks. *Internet Things* **2021**, *16*, 100462. [\[CrossRef\]](#)
15. Zamani, M.; Movahedi, M. Machine learning techniques for intrusion detection. *arXiv* **2013**, arXiv:1312.2177.
16. Kim, J.; Shin, N.; Jo, S.Y.; Kim, S.H. Method of intrusion detection using deep neural network. In Proceedings of the 2017 IEEE International Conference on Big Data and Smart Computing (BigComp), Jeju, Republic of Korea, 13–16 February 2017; IEEE: New York, NY, USA, 2017; pp. 313–316.
17. Tang, C.; Luktarhan, N.; Zhao, Y. SAAE-DNN: Deep learning method on intrusion detection. *Symmetry* **2020**, *12*, 1695. [\[CrossRef\]](#)
18. Tao, P.; Sun, Z.; Sun, Z. An improved intrusion detection algorithm based on GA and SVM. *IEEE Access* **2018**, *6*, 13624–13631. [\[CrossRef\]](#)
19. Deng, H.; Zeng, Q.A.; Agrawal, D.P. SVM-based intrusion detection system for wireless ad hoc networks. In Proceedings of the 2003 IEEE 58th Vehicular Technology Conference. VTC 2003-Fall (IEEE Cat. No. 03CH37484), Orlando, FL, USA, 6–9 October 2003; IEEE: New York, NY, USA, 2003; Volume 3, pp. 2147–2151.
20. Ingre, B.; Yadav, A.; Soni, A.K. Decision tree based intrusion detection system for NSL-KDD dataset. In *Information and Communication Technology for Intelligent Systems (ICTIS 2017)-Volume 2, Proceedings of the ICTIS 2017: Second International Conference on Information and Communication Technology for Intelligent Systems, Ahmedabad, India, 25–26 March 2017*; Springer: Berlin/Heidelberg, Germany, 2018; pp. 207–218.
21. Ferrag, M.A.; Maglaras, L.; Ahmim, A.; Derdour, M.; Janicke, H. Rdtids: Rules and decision tree-based intrusion detection system for internet-of-things networks. *Future Internet* **2020**, *12*, 44. [\[CrossRef\]](#)
22. Al-Omari, M.; Rawashdeh, M.; Qutaishat, F.; Alshira’H, M.; Ababneh, N. An intelligent tree-based intrusion detection model for cyber security. *J. Netw. Syst. Manag.* **2021**, *29*, 20. [\[CrossRef\]](#)
23. Amor, N.B.; Benferhat, S.; Elouedi, Z. Naive bayes vs decision trees in intrusion detection systems. In Proceedings of the 2004 ACM Symposium on Applied Computing, Nicosia, Cyprus, 14–17 March 2004; pp. 420–424.
24. Panigrahi, R.; Borah, S.; Pramanik, M.; Bhoi, A.K.; Barsocchi, P.; Nayak, S.R.; Alnumay, W. Intrusion detection in cyber-physical environment using hybrid Naïve Bayes—Decision table and multi-objective evolutionary feature selection. *Comput. Commun.* **2022**, *188*, 133–144. [\[CrossRef\]](#)
25. Balyan, A.K.; Ahuja, S.; Lilhore, U.K.; Sharma, S.K.; Manoharan, P.; Algarni, A.D.; Elmannai, H.; Raahemifar, K. A hybrid intrusion detection model using ega-pso and improved random forest method. *Sensors* **2022**, *22*, 5986. [\[CrossRef\]](#)
26. Waskle, S.; Parashar, L.; Singh, U. Intrusion detection system using PCA with random forest approach. In Proceedings of the 2020 International Conference on Electronics and Sustainable Communication Systems (ICESC), Coimbatore, India, 2–4 July 2020; IEEE: New York, NY, USA, 2020; pp. 803–808.
27. Negandhi, P.; Trivedi, Y.; Mangrulkar, R. Intrusion detection system using random forest on the NSL-KDD dataset. In *Emerging Research in Computing, Information, Communication and Applications*; Springer: Berlin/Heidelberg, Germany, 2019; pp. 519–531.
28. Das, A.; Rad, P. Opportunities and challenges in explainable artificial intelligence (xai): A survey. *arXiv* **2020**, arXiv:2006.11371.
29. Bakro, M.; Kumar, R.R.; Husain, M.; Ashraf, Z.; Ali, A.; Yaqoob, S.I.; Ahmed, M.N.; Parveen, N. Building a Cloud-IDS by Hybrid Bio-Inspired Feature Selection Algorithms Along With Random Forest Model. *IEEE Access* **2024**, *12*, 8846–8874. [\[CrossRef\]](#)
30. Dhupal, C.T.; Pingale, S.V. Analysis of Intrusion Detection Systems: Techniques, Datasets and Research Opportunity. *SSRN Electron. J.* **2024**, 1–8. [\[CrossRef\]](#)
31. Khan, A.R.; Kashif, M.; Jhaveri, R.H.; Raut, R.; Saba, T.; Bahaj, S.A. Deep Learning for Intrusion Detection and Security of Internet of Things (IoT): Current Analysis, Challenges, and Possible Solutions. *Secur. Commun. Netw.* **2022**, *2022*, 4016073. [\[CrossRef\]](#)

32. Mesadieu, F.; Torre, D.; Chennameneni, A. Leveraging Deep Reinforcement Learning Technique for Intrusion Detection in SCADA Infrastructure. *IEEE Access* **2024**, *12*, 63381–63399. [\[CrossRef\]](#)
33. Panigrahi, R.; Borah, S. A detailed analysis of CICIDS2017 dataset for designing Intrusion Detection Systems. *Int. J. Eng. Technol.* **2018**, *7*, 479–482.
34. Dhanabal, L.; Shantharajah, S. A study on NSL-KDD dataset for intrusion detection system based on classification algorithms. *Int. J. Adv. Res. Comput. Commun. Eng.* **2015**, *4*, 446–452.
35. Mihailescu, M.E.; Mihai, D.; Carabas, M.; Komisarek, M.; Pawlicki, M.; Hołubowicz, W.; Kozik, R. The Proposition and Evaluation of the RoEduNet-SIMARGL2021 Network Intrusion Detection Dataset. *Sensors* **2021**, *21*, 4319. [\[CrossRef\]](#)
36. Stiawan, D.; Idris, M.Y.; Bamhdi, A.M.; Budiarto, R. CICIDS-2017 Dataset Feature Analysis With Information Gain for Anomaly Detection. *IEEE Access* **2020**, *8*, 132911–132921. [\[CrossRef\]](#)
37. Warnecke, A.; Arp, D.; Wressnegger, C.; Rieck, K. Evaluating explanation methods for deep learning in security. In Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P), Genoa, Italy, 7–11 September 2020; IEEE: New York, NY, USA, 2020; pp. 158–174.
38. Strom, B.E.; Applebaum, A.; Miller, D.P.; Nickels, K.C.; Pennington, A.G.; Thomas, C.B. Mitre att&ck: Design and philosophy. In *Technical Report*; The MITRE Corporation: McLean, VA, USA, 2018.
39. Lee, C.B.; Roedel, C.; Silenok, E. *Detection and Characterization of Port Scan Attacks*; University of California, Department of Computer Science and Engineering: San Diego, CA, USA, 2003.
40. Comprmoise, D. Drive-By Compromise. 2023. Available online: <https://attack.mitre.org/techniques/T1189/> (accessed on 21 October 2023).
41. Chen, Y.; Lin, Q.; Wei, W.; Ji, J.; Wong, K.C.; Coello Coello, C.A. Intrusion detection using multi-objective evolutionary convolutional neural network for Internet of Things in Fog computing. *Knowl.-Based Syst.* **2022**, *244*, 108505. [\[CrossRef\]](#)
42. Stone-Gross, B.; Cova, M.; Cavallaro, L.; Gilbert, B.; Szydlowski, M.; Kemmerer, R.; Kruegel, C.; Vigna, G. Your botnet is my botnet: Analysis of a botnet takeover. In Proceedings of the 16th ACM Conference on Computer and Communications Security, Chicago, IL, USA, 9–13 November 2009; pp. 635–647.
43. Gorodetski, V.; Kutenko, I. Attacks against computer network: Formal grammar-based framework and simulation tool. In *International Workshop on Recent Advances in Intrusion Detection, Proceedings of the 5th International Symposium, RAID 2002, Zurich, Switzerland, 16–18 October 2002*; Springer: Berlin/Heidelberg, Germany, 2002; pp. 219–238.
44. Khan, A.; Kim, H.; Lee, B. M2MON: Building an MMIO-based Security Reference Monitor for Unmanned Vehicles. In Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), Virtual, 11–13 August 2021.
45. Hussain, S.R.; Karim, I.; Ishtiaq, A.A.; Chowdhury, O.; Bertino, E. Noncompliance as deviant behavior: An automated black-box noncompliance checker for 4g lte cellular devices. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual, 15–19 November 2021; pp. 1082–1099.
46. Mirzaei, O.; Vasilenko, R.; Kirda, E.; Lu, L.; Kharraz, A. Scrutinizer: Detecting code reuse in malware via decompilation and machine learning. In Proceedings of the Detection of Intrusions and Malware, and Vulnerability Assessment: 18th International Conference, DIMVA 2021, Virtual Event, 14–16 July 2021; Proceedings 18; Springer: Berlin/Heidelberg, Germany, 2021; pp. 130–150.
47. Lukacs, S.; Sirb, C.B.; Lutas, D.H.; Colesa, A.V. Strongly Isolated Malware Scanning Using Secure Virtual Containers. U.S. Patent 9,117,081, 25 August 2015.
48. Kim, A.; Park, M.; Lee, D.H. AI-IDS: Application of deep learning to real-time Web intrusion detection. *IEEE Access* **2020**, *8*, 70245–70261. [\[CrossRef\]](#)
49. Botacin, M.; Ceschin, F.; Sun, R.; Oliveira, D.; Grégio, A. Challenges and pitfalls in malware research. *Comput. Secur.* **2021**, *106*, 102287. [\[CrossRef\]](#)
50. Lundberg, S.M.; Lee, S.I. A unified approach to interpreting model predictions. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 4768–4777.
51. Dieber, J.; Kirrane, S. Why model why? Assessing the strengths and limitations of LIME. *arXiv* **2020**, arXiv:2012.00093.
52. DCunha, S.D. Is AI Shifting the Human-in-the-Loop Model in Cybersecurity? 2017. Available online: <https://datatechvibe.com/ai/is-ai-shifting-the-human-in-the-loop-model-in-cybersecurity/> (accessed on 21 October 2021).
53. Han, D.; Wang, Z.; Chen, W.; Zhong, Y.; Wang, S.; Zhang, H.; Yang, J.; Shi, X.; Yin, X. DeepAID: Interpreting and Improving Deep Learning-based Anomaly Detection in Security Applications. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual, 15–19 November 2021.
54. Wu, C.; Qian, A.; Dong, X.; Zhang, Y. Feature-oriented Design of Visual Analytics System for Interpretable Deep Learning based Intrusion Detection. In Proceedings of the 2020 International Symposium on Theoretical Aspects of Software Engineering (TASE), Hangzhou, China, 11–13 December 2020; pp. 73–80. [\[CrossRef\]](#)
55. Mirsky, Y.; Doitshman, T.; Elovici, Y.; Shabtai, A. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. *arXiv* **2018**, arXiv:1802.09089.
56. Liaw, A.; Wiener, M. Classification and regression by randomForest. *R News* **2002**, *2*, 18–22.
57. Aggarwal, C.C.; Hinneburg, A.; Keim, D.A. On the surprising behavior of distance metrics in high dimensional space. In Proceedings of the Database Theory—ICDT 2001: 8th International Conference, London, UK, 4–6 January 2001; Proceedings 8; Springer: Berlin/Heidelberg, Germany, 2001; pp. 420–434.

58. Chicco, D.; Jurman, G. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genom.* **2020**, *21*, 6. [CrossRef] [PubMed]
59. Salih, A.; Raisi-Estabragh, Z.; Galazzo, I.B.; Radeva, P.; Petersen, S.E.; Menegaz, G.; Lekadir, K. Commentary on explainable artificial intelligence methods: SHAP and LIME. *arXiv* **2023**, arXiv:2305.02012.
60. Garreau, D.; von Luxburg, U. Explaining the Explainer: A First Theoretical Analysis of LIME. *arXiv* **2020**, arXiv:2001.03447.
61. Ma, X.; Hou, M.; Zhan, J.; Liu, Z. Interpretable Predictive Modeling of Tight Gas Well Productivity with SHAP and LIME Techniques. *Energies* **2023**, *16*, 3653. [CrossRef]
62. Roberts, C.V.; Elahi, E.; Chandrashekar, A. On the Bias-Variance Characteristics of LIME and SHAP in High Sparsity Movie Recommendation Explanation Tasks. *arXiv* **2022**, arXiv:2206.04784.
63. Flow Information Elements—Nprobe 10.1 Documentation. Available online: https://www.ntop.org/guides/nprobe/flow_information_elements.html (accessed on 9 April 2024).
64. Ahlaskari. CICFLOWMETER/README.TXT at Master · Ahlaskari/Cicflowmeter. 2021. Available online: <https://github.com/ahlashkari/CICFlowMeter/blob/master/ReadMe.txt> (accessed on 9 April 2024).
65. Claise, B. Cisco Systems Netflow Services Export Version 9; Technical Report. Available online: https://www.cisco.com/en/US/technologies/tk648/tk362/technologies_white_paper09186a00800a3db9.html (accessed on 9 April 2024).
66. Sharafaldin, I.; Gharib, A.; Lashkari, A.H.; Ghorbani, A.A. Towards a reliable intrusion detection benchmark dataset. *Softw. Netw.* **2018**, *2018*, 177–200. [CrossRef]
67. Tavallaee, M.; Bagheri, E.; Lu, W.; Ghorbani, A.A. A detailed analysis of the KDD CUP 99 data set. In Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, Ottawa, ON, Canada, 8–10 July 2009; pp. 1–6. [CrossRef]
68. Zhao, R. NetFlow Version 9 Flow-Record Format. Available online: <https://ieee-dataport.org/documents/nsf-kdd-0> (accessed on 9 April 2024).
69. Brownlee, J. How to Choose a Feature Selection Method for Machine Learning. Available online: <https://machinelearningmastery.com/feature-selection-with-real-and-categorical-data/> (accessed on 9 April 2024).
70. Neupane, S.; Ables, J.; Anderson, W.; Mittal, S.; Rahimi, S.; Banicescu, I.; Seale, M. Explainable Intrusion Detection Systems (X-IDS): A Survey of Current Methods, Challenges, and Opportunities. *arXiv* **2022**, arXiv:2207.06236.
71. Dwivedi, R.; Dave, D.; Naik, H.; Singhal, S.; Omer, R.; Patel, P.; Qian, B.; Wen, Z.; Shah, T.; Morgan, G.; et al. Explainable AI (XAI): Core Ideas, Techniques, and Solutions. *ACM Comput. Surv.* **2023**, *55*, 1–33. [CrossRef]
72. Mahbooba, B.; Timilsina, M.; Sahal, R.; Serrano, M. Explainable artificial intelligence (XAI) to enhance trust management in intrusion detection systems using decision tree model. *Complexity* **2021**, *2021*, 6634811. [CrossRef]
73. Patil, S.; Varadarajan, V.; Mazhar, S.M.; Sahibzada, A.; Ahmed, N.; Sinha, O.; Kumar, S.; Shaw, K.; Kotecha, K. Explainable Artificial Intelligence for Intrusion Detection System. *Electronics* **2022**, *11*, 3079. [CrossRef]
74. Islam, S.R.; Eberle, W.; Ghafoor, S.K.; Siraj, A.; Rogers, M. Domain knowledge aided explainable artificial intelligence for intrusion detection and response. *arXiv* **2019**, arXiv:1911.09853.
75. Roponen, E.; Kampars, J.; Grabis, J.; Gailītis, A. Towards a Human-in-the-Loop Intelligent Intrusion Detection System. In Proceedings of the CEUR Workshop Proceedings, Trier, Germany, 19 September 2022; pp. 71–81.
76. Chaddad, A.; Peng, J.; Xu, J.; Bouridane, A. Survey of Explainable AI Techniques in Healthcare. *Sensors* **2023**, *23*, 634. [CrossRef]
77. Nwakanma, C.I.; Ahakonye, L.A.C.; Njoku, J.N.; Odirichukwu, J.C.; Okolie, S.A.; Uzundu, C.; Ndubuisi Nweke, C.C.; Kim, D.S. Explainable Artificial Intelligence (XAI) for Intrusion Detection and Mitigation in Intelligent Connected Vehicles: A Review. *Appl. Sci.* **2023**, *13*, 1252. [CrossRef]
78. Nazat, S.; Li, L.; Abdallah, M. XAI-ADS: An Explainable Artificial Intelligence Framework for Enhancing Anomaly Detection in Autonomous Driving Systems. *IEEE Access* **2024**, *12*, 48583–48607. [CrossRef]
79. Ram, A.; Veni, K.; Zeadally, S. An intelligent SDN-IoT enabled intrusion detection system for healthcare systems using a hybrid deep learning and machine learning approach. *China Commun.* **2024**, 1–21. [CrossRef]
80. Zebin, T.; Rezvy, S.; Luo, Y. An Explainable AI-Based Intrusion Detection System for DNS Over HTTPS (DoH) Attacks. *IEEE Trans. Inf. Forensics Secur.* **2022**, *17*, 2339–2349. [CrossRef]
81. Han, D.; Wang, Z.; Chen, W.; Wang, K.; Yu, R.; Wang, S.; Zhang, H.; Wang, Z.; Jin, M.; Yang, J.; et al. Anomaly Detection in the Open World: Normality Shift Detection, Explanation, and Adaptation. In Proceedings of the 30th Annual Network and Distributed System Security Symposium (NDSS), San Diego, CA, USA, 27 February–3 March 2023.
82. Wang, M.; Zheng, K.; Yang, Y.; Wang, X. An Explainable Machine Learning Framework for Intrusion Detection Systems. *IEEE Access* **2020**, *8*, 73127–73141. [CrossRef]
83. Arreche, O.; Guntur, T.R.; Roberts, J.W.; Abdallah, M. E-XAI: Evaluating Black-Box Explainable AI Frameworks for Network Intrusion Detection. *IEEE Access* **2024**, *12*, 23954–23988. [CrossRef]
84. Alazab, A.; Hobbs, M.; Abawajy, J.; Alazab, M. Using feature selection for intrusion detection system. In Proceedings of the 2012 International Symposium on Communications and Information Technologies (ISCIT), Gold Coast, Australia, 2–5 October 2012; IEEE: New York, NY, USA, 2012; pp. 296–301.
85. Mebawondu, J.O.; Alowolodu, O.D.; Mebawondu, J.O.; Adetunmbi, A.O. Network intrusion detection system using supervised learning paradigm. *Sci. Afr.* **2020**, *9*, e00497. [CrossRef]

86. Kamalov, F.; Moussa, S.; Zgheib, R.; Mashaal, O. Feature selection for intrusion detection systems. In Proceedings of the 2020 13th International Symposium on Computational Intelligence and Design (ISCID), Hangzhou, China, 12–13 December 2020; IEEE: New York, NY, USA, 2020; pp. 265–269.
87. Li, X.; Yi, P.; Wei, W.; Jiang, Y.; Tian, L. LNNLS-KH: A feature selection method for network intrusion detection. *Secur. Commun. Netw.* **2021**, 2021, 8830431. [[CrossRef](#)]
88. Guo, W.; Mu, D.; Xu, J.; Su, P.; Wang, G.; Xing, X. LEMNA: Explaining Deep Learning Based Security Applications. In Proceedings of the CCS '18, 2018 ACM SIGSAC Conference on Computer and Communications Security, Toronto, ON, Canada, 15–19 October 2018; pp. 364–379. [[CrossRef](#)]
89. Yang, L.; Guo, W.; Hao, Q.; Ciptadi, A.; Ahmadzadeh, A.; Xing, X.; Wang, G. CADE: Detecting and Explaining Concept Drift Samples for Security Applications. In Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), Virtual, 11–13 August 2021; USENIX Association: Berkeley, CA, USA, 2021; pp. 2327–2344.
90. Charmet, F.; Tanuwidjaja, H.C.; Ayoubi, S.; Gimenez, P.F.; Han, Y.; Jmila, H.; Blanc, G.; Takahashi, T.; Zhang, Z. Explainable artificial intelligence for cybersecurity: A literature survey. *Ann. Telecommun.* **2022**, 77, 789–812. [[CrossRef](#)]
91. Insights, I. 42 Cyber Attack Statistics by Year: A Look at the Last Decade. 2020. Available online: <https://sectigostore.com/blog/42-cyber-attack-statistics-by-year-a-look-at-the-last-decade/> (accessed on 10 March 2023).
92. Moustafa, N.; Slay, J. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), Canberra, Australia, 10–12 November 2015; IEEE: New York, NY, USA, 2015; pp. 1–6.
93. CSE-CIC-IDS2018. CSE-CIC-IDS2018 on AWS. 2018. Available online: <https://www.unb.ca/cic/datasets/ids-2018.html/> (accessed on 10 November 2023).
94. Garcia, S.; Grill, M.; Stiborek, J.; Zunino, A. CTU-13: Botnet Detection from Network Traffic, StratosphereLab@CTU Prague. Available online: <https://www.kaggle.com/datasets/dhoogla/ctu13> (accessed on 9 April 2024).
95. Repository, U.T. UMass Trace Repository. 2021. Available online: <http://traces.cs.umass.edu/index.php/Network/Network> (accessed on 21 November 2022).
96. Dong, Y.; Guo, W.; Chen, Y.; Xing, X.; Zhang, Y.; Wang, G. Towards the Detection of Inconsistencies in Public Security Vulnerability Reports. In Proceedings of the 28th USENIX Security Symposium (USENIX Security 19), Santa Clara, CA, USA, 14–16 August 2019; pp. 869–885.
97. Hoffman, R.R.; Mueller, S.T.; Klein, G.; Litman, J. Metrics for Explainable AI: Challenges and Prospects. *arXiv* **2019**, arXiv:1812.04608.
98. Sundararajan, M.; Taly, A.; Yan, Q. Axiomatic attribution for deep networks. In Proceedings of the International Conference on Machine Learning. PMLR, Sydney, Australia, 6–11 August 2017; pp. 3319–3328.
99. D’hooge, L.; Verkerken, M.; Volckaert, B.; Wauters, T.; De Turck, F. Establishing the Contaminating Effect of Metadata Feature Inclusion in Machine-Learned Network Intrusion Detection Models. In Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, Cagliari, Italy, 29 June–1 July 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 23–41.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.